

# 中国科学技术大学

# 硕士学位论文



## 基于融合特征的加密流量分类方法 研究

作者姓名： 符志鹏

学科专业： 计算机科学与技术

导 师： 曾凡平 副教授

完成时间： 二〇二六年三月二十五日



University of Science and Technology of China

A dissertation for master's degree



# **Research on Encryption Traffic Classification Method Based on Fusion Features**

Author: Zhipeng Fu

Speciality: Computer Science and Technology

Supervisor: Assoc. Prof. Fanping Zeng

Completion date: March 25, 2026



## 中国科学技术大学学位论文原创性声明

本人声明所呈交的学位论文，是本人在导师指导下进行研究工作所取得的成果。除已特别加以标注和致谢的地方外，论文中不包含任何他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的贡献均已在论文中作了明确的说明。

作者签名：\_\_\_\_\_

签字日期：\_\_\_\_\_

## 中国科学技术大学学位论文授权使用声明

作为申请学位的条件之一，学位论文著作权拥有者授权中国科学技术大学拥有学位论文的部分使用权，即：学校有权按有关规定向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅，可以将学位论文编入《中国学位论文全文数据库》等有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编学位论文。本人提交的电子文档的内容和纸质论文的内容相一致。

控阅的学位论文在解除后也遵守此规定。

☒ 公开   ☐ 控阅（\_\_\_\_年）

作者签名：\_\_\_\_\_

导师签名：\_\_\_\_\_

签字日期：\_\_\_\_\_

签字日期：\_\_\_\_\_



## 摘要

识别流量类型是网络研究的一项基本且关键的能力，对服务质量控制、安全检测等应用至关重要。随着用户隐私意识增强，SSL/TLS、VPN 和 Tor 等加密工具普及（如美国加密流量占比从 2017 年 55% 升至 2025 年 94%），传统基于端口和深度包检测（DPI）的方法因无法解析加密的数据包载荷而失效。机器学习方法虽利用流统计特征实现分类，但依赖人工特征工程，泛化能力有限且实施成本高。深度学习技术通过端到端学习范式突破上述限制：可直接处理原始流量数据，自动提取特征，支持早期分类，并减少对专家知识的依赖。然而，现有深度学习方案多存在显著局限：仅根据流级别或包级别特征来分析和识别流量，而忽略了另一维度所包含的流量特征，未能融合双维度信息；并且，在包级别的处理中，还存在包头和有效载荷的混合使用的情况，既未区分功能差异，又因使用有效载荷数据而损害用户的隐私权益。因此，研究一种融合流级别与包级别特征，且仅使用 IP 数据包头，不涉及数据包有效载荷的基于深度学习的融合加密流量分类方法，对提升网络安全和服务质量具有重要价值。

本文的主要工作内容如下：

(1) 在本文中，我们提出了一种基于 BYOL 模型的流级别特征分类方法。我们的目的是仅使用数据流的早期的数据包长度序列，就能够实现有效的早期流量分类。该方案通过对数据包长度序列进行截取，然后对同一个长度序列进行两种差异化增强，然后分别输入 BYOL 模型的在线网络和目标网络，从而提取出两种增强变体之间不变的，具有鲁棒性的流级别特征。实验表明，与不具有对比学习机制的传统方法相比，我们的方法在多个加密流量数据集上实现了更高的分类准确率。

(2) 为了进一步提升分类效果，本文设计了一种基于 n-gram 技术的包级别特征提取方法，仅使用 IP 数据包头，而不涉及数据包的有效载荷，通过对 IP 包头数据进行不同窗口大小的结合，有效地学习了包头字段的字节组合与顺序特征。并且通过动态权重的特征融合机制，将流级别和包级别的特征进行融合，实现了特征间非线性关系的有效建模，并在多个加密流量数据集上进行了验证。实验结果表明，融合了流级别和包级别特征的分类方法显著优于仅使用单一维度特征的方法，进一步提升了加密流量分类的准确率。

**关键词：**加密流量；流量分类；自监督学习；特征融合

## ABSTRACT

Identifying traffic types is a fundamental and crucial capability in network research, essential for applications such as Quality of Service (QoS) control and security testing. With increasing user privacy awareness and the widespread adoption of encryption tools such as SSL/TLS, VPNs, and Tor (e.g., the percentage of encrypted traffic in the US is projected to rise from 55% in 2017 to 94% in 2025), traditional port-based and deep packet inspection (DPI) methods become ineffective due to their inability to parse encrypted packet payloads. While machine learning methods utilize flow statistics for classification, they rely on manual feature engineering, resulting in limited generalization capabilities and high implementation costs. Deep learning technology overcomes these limitations through an end-to-end learning paradigm: it can directly process raw traffic data, automatically extract features, support early classification, and reduce reliance on expert knowledge. However, existing deep learning solutions often have significant limitations: they analyze and identify traffic based solely on flow-level or packet-level features, neglecting traffic features contained in the other dimension and failing to integrate dual-dimensional information. Furthermore, in packet-level processing, there is a situation where headers and payloads are used interchangeably, failing to distinguish functional differences and infringing on user privacy due to the use of payload data. Therefore, researching a deep learning-based fusion encrypted traffic classification method that integrates flow-level and packet-level features, using only IP packet headers and excluding packet payloads, is of significant value for improving network security and service quality.

The main contributions of this paper are as follows:

(1) In this paper, we propose a flow-level feature classification method based on the BYOL model. Our goal is to achieve effective early traffic classification using only the early packet length sequence of the data flow. This scheme extracts robust flow-level features invariant between the two enhancement variants by truncating the packet length sequence and then applying two differential enhancements to the same length sequence, which are then input into the online and target networks of the BYOL model, respectively. Experiments show that our method achieves higher classification accuracy on multiple encrypted traffic datasets compared to traditional methods without a contrastive learning mechanism.

(2) To further improve classification performance, this paper designs a packet-level

feature extraction method based on n-gram technology. This method uses only the IP packet header, excluding the packet payload. By combining IP packet header data with different window sizes, it effectively learns the byte combination and order features of the header fields. Furthermore, through a dynamic weighted feature fusion mechanism, flow-level and packet-level features are fused, effectively modeling the nonlinear relationship between features. This method has been validated on multiple encrypted traffic datasets. Experimental results show that the classification method fusing flow-level and packet-level features significantly outperforms methods using only single-dimensional features, further improving the accuracy of encrypted traffic classification.

**KEY WORDS:** Encrypted traffic, Traffic classification, Self-Supervised Learning, Feature fusion

# 目录

|                                        |           |
|----------------------------------------|-----------|
| <b>第 1 章 绪论 .....</b>                  | <b>1</b>  |
| 1.1 研究背景及意义 .....                      | 1         |
| 1.2 国内外研究现状 .....                      | 2         |
| 1.2.1 基于载荷检测的方法 .....                  | 2         |
| 1.2.2 基于传统机器学习的方法 .....                | 3         |
| 1.2.3 基于深度学习的方法 .....                  | 4         |
| 1.3 研究内容 .....                         | 5         |
| 1.4 本文主要贡献 .....                       | 6         |
| 1.5 章节安排 .....                         | 6         |
| <b>第 2 章 相关理论概述 .....</b>              | <b>7</b>  |
| 2.1 网络流量分类理论 .....                     | 7         |
| 2.1.1 端口检测 .....                       | 7         |
| 2.1.2 深度包检测 .....                      | 8         |
| 2.1.3 传统机器学习方法 .....                   | 8         |
| 2.2 深度学习概述 .....                       | 9         |
| 2.2.1 卷积神经网络 .....                     | 9         |
| 2.2.2 激活函数 .....                       | 12        |
| 2.2.3 损失函数 .....                       | 14        |
| 2.2.4 残差神经网络 .....                     | 15        |
| 2.2.5 注意力机制 .....                      | 16        |
| 2.3 自监督学习 .....                        | 17        |
| 2.3.1 对比学习 .....                       | 17        |
| 2.3.2 数据增强 .....                       | 18        |
| 2.4 本章小结 .....                         | 18        |
| <b>第 3 章 基于 BYOL 模型的流级特征分类方法 .....</b> | <b>19</b> |
| 3.1 系统模型 .....                         | 19        |
| 3.2 增强函数 .....                         | 20        |
| 3.2.1 增强函数 1: 噪声注入 .....               | 20        |
| 3.2.2 增强函数 2: 尺度缩放与随机遮蔽 .....          | 21        |
| 3.3 模型架构 .....                         | 22        |
| 3.3.1 编码器 .....                        | 22        |

|              |                          |           |
|--------------|--------------------------|-----------|
| 3.3.2        | 投影器和预测器 .....            | 23        |
| 3.3.3        | 损失函数 .....               | 24        |
| 3.3.4        | 指数移动平均值 (EMA) 更新机制 ..... | 25        |
| 3.4          | 训练优化 .....               | 25        |
| 3.4.1        | 学习率调度与 AdamW 优化 .....    | 25        |
| 3.4.2        | 针对抖动的回溯策略 .....          | 25        |
| 3.5          | 实验设计与结果分析 .....          | 26        |
| 3.5.1        | 实验环境 .....               | 26        |
| 3.5.2        | 数据集介绍与预处理 .....          | 26        |
| 3.5.3        | 评价指标 .....               | 29        |
| 3.5.4        | 参数设置 .....               | 29        |
| 3.5.5        | 对比算法 .....               | 31        |
| 3.5.6        | 实验结果分析 .....             | 31        |
| 3.6          | 本章小结 .....               | 33        |
| <b>第 4 章</b> | <b>基于融合特征的分类方法 .....</b> | <b>34</b> |
| 4.1          | 系统模型 .....               | 35        |
| 4.2          | 包级别特征提取 .....            | 35        |
| 4.2.1        | 报文首部截取与预处理 .....         | 35        |
| 4.2.2        | N-gram 嵌入处理 .....        | 36        |
| 4.3          | 交叉注意力特征融合机制 .....        | 38        |
| 4.3.1        | 线性投影与空间对齐 .....          | 38        |
| 4.3.2        | 交叉注意力流 .....             | 38        |
| 4.4          | 深度残差分类器 .....            | 40        |
| 4.4.1        | 输入层 .....                | 41        |
| 4.4.2        | 深层残差主干网络 .....           | 41        |
| 4.4.3        | 输出层逻辑 .....              | 42        |
| 4.5          | 训练优化 .....               | 42        |
| 4.5.1        | 学习率调度 .....              | 42        |
| 4.6          | 实验设计与结果分析 .....          | 42        |
| 4.6.1        | 参数设置 .....               | 43        |
| 4.6.2        | 对比算法 .....               | 43        |
| 4.6.3        | 实验结果分析 .....             | 44        |
| 4.7          | 本章小结 .....               | 46        |

|                      |    |
|----------------------|----|
| 第 5 章 总结与展望 .....    | 47 |
| 5.1 本文工作总结 .....     | 47 |
| 5.2 未来展望 .....       | 47 |
| 第 6 章 引用文献的标注 .....  | 48 |
| 6.1 顺序编码制 .....      | 48 |
| 6.1.1 角标数字标注法 .....  | 48 |
| 6.1.2 数字标注法 .....    | 48 |
| 6.2 著者-出版年制标注法 ..... | 48 |
| 致谢 .....             | 49 |
| 在读期间取得的科研成果 .....    | 50 |

## 插图和附表清单

|       |                     |    |
|-------|---------------------|----|
| 图 2.1 | 卷积神经网络结构 .....      | 10 |
| 图 2.2 | 卷积运算过程 .....        | 11 |
| 图 2.3 | 两种池化方式 .....        | 12 |
| 图 2.4 | 常见的激活函数图像 .....     | 13 |
| 图 2.5 | 注意力机制处理过程 .....     | 16 |
| 图 2.6 | 对比学习过程 .....        | 17 |
| 图 3.1 | BYOL 模型框架 .....     | 19 |
| 图 3.2 | 数据包序列变化 .....       | 20 |
| 图 3.3 | 编码器结构 .....         | 22 |
| 图 3.4 | 数据预处理流程 .....       | 28 |
| 图 4.1 | 两个混淆矩阵 .....        | 34 |
| 图 4.2 | n-gram 嵌入处理 .....   | 37 |
| 图 4.3 | 交叉注意力机制 .....       | 39 |
| 图 4.4 | 分类器结构 .....         | 40 |
| 图 4.5 | 三种模型的混淆矩阵 .....     | 46 |
| 表 2.1 | 常见激活函数的公式 .....     | 13 |
| 表 2.2 | 常见损失函数的公式 .....     | 15 |
| 表 3.1 | 实验环境设置 .....        | 26 |
| 表 3.2 | 数据集内容 .....         | 27 |
| 表 3.3 | 模型及训练参数设置 .....     | 30 |
| 表 3.4 | 不同数据集上的对比实验结果 ..... | 32 |
| 表 4.1 | IP 数据报头结构 .....     | 36 |
| 表 4.2 | 模型及训练参数设置 .....     | 43 |
| 表 4.3 | 不同数据集上的对比实验结果 ..... | 45 |



## 第1章 绪论

### 1.1 研究背景及意义

识别流量类型是网络研究的一项基本且关键的能力。流量分类,即将网络流量归类到适当的类别中,对许多方面的应用至关重要,例如服务质量(QoS)控制、定价、资源使用规划、恶意软件检测和入侵检测。

随着安全意识的觉醒,互联网用户认识到隐私保护的重要性,并逐渐使用加密的网络通信工具或协议进行网络访问。根据 Firefox 的研究统计,美国用户利用 SSL/TLS 安全协议的百分比从 2017 年的 55% 上升到了 2024 年的 94%[1]。此外,虚拟专用网络(VPN)和洋葱路由器(Tor)也被用户广泛采用,以确保网络通信的匿名性。在此背景下,加密流量分类被视为保障服务质量(QoS)、体验质量(QoE)和网络安全供给的关键技术。

早期的流量分类方法包括基于端口的技术和深度包检测(DPI)[2-4]。基于端口的识别方法利用端口与协议之间的对应关系进行流量分类,但由于端口转发和动态端口协商技术的广泛使用而变得无效。DPI 通过分析整个 IP 数据包来识别特定协议和应用数据。由于它无法解析加密的数据包载荷,因此无法实现加密流量分类。

人工智能技术的进步开启了流量分类方法的新时代,主要围绕着机器学习和深度学习算法的应用。基于机器学习的方法通常利用流量的统计模式构建流量特征,例如数据包路径签名、数据包到达时间和大小以及累积数据包大小,然后选择性能最佳的分类器对其进行分类。基于机器学习方法的研究主题同时也是其缺陷在于,必须完成特征工程,即为每个特定的加密流量协议或工具选择最优特征。这项工作高度依赖于领域专业知识,且所选特征通常泛化能力较差。在真实的网络场景中,为不同工具或协议生成的各类加密流量训练和实施不同的基于机器学习的加密流量分类方法既耗时又难以实现。

作为机器学习的一个分支,深度学习在加密流量分类领域已展现出巨大的潜力。深度学习技术的优势在于模型可以直接输入原始流量数据,并在监督训练过程中自动执行特征提取和表征,从而消除了机器学习中对特征工程和专家知识的需求。基于深度学习的方法天然适用于早期的加密流量分类场景(即在流量会话早期进行分类),这与依赖于完整流或会话的基于机器学习的方法形成鲜明对比。深度学习采用端到端的学习范式,直接从原始输入数据学习以产生期望的输出,绕过了将问题分解为特征选择和分类等子问题的传统做法。这一特性使深度学习算法能够全面应对与流量分类任务相关的复杂性。

然而,当前存在的算法中的大多数是简单的神经网络,仅根据流级或包级特征来分析和识别流量,而忽略了另一维度所包含的流量特征。此处的流级特征是从流的包长度序列中提取的,而包级特征则是从流的数据包中提取的。此外,对于包级别特征,还存在包头和有效载荷的混合使用的情况,将包头和有效载荷视为一个统一整体进行学习,没有区分它们不同的表示和功能。使用到有效载荷数据还存在的一个问题是,这不可避免地违反了用户的隐私保护要求。

综上所述,基于深度学习的加密流量分类方法能同时满足有效分类与高泛用性的要求,但当前多数方法仍存在某些不足。因此,研究一种融合流级别与包级别特征、不依赖有效载荷数据的基于深度学习的融合加密流量分类方法,具有很高的研究价值和意义。

## 1.2 国内外研究现状

随着互联网信息化和数字化的高速发展,网络流量分类近年来已成为一个热门的研究领域。在过去的几十年间,研究者提出了许多方法和技术。本文将从基于载荷检测的方法、基于传统机器学习的方法以及基于深度学习的方法三个方面介绍国内外的研究概况。

### 1.2.1 基于载荷检测的方法

基于载荷检测的流量分类方法,也称为深度包检测(DPI),通过将数据包载荷与一组已知的协议特征签名(例如,HTTP流量中的GET签名)进行比对来工作[5]。一些著名的DPI库包括libprotoident、OpenDPI和nDPI。nDPI包含许多知名协议(例如HTTP和FTP)的专用协议解码器[6]。此外,一些基于载荷检测的方法也会检查数据包头部的字段值来识别网络流量,其中最常用的字段是端口号。例如,端口为80的数据包很可能首先被HTTP解码器解析。然而,这种方法在端口动态分配和通用通信协议端口的情况下会失效[7-8]。

DPI是一种计算开销较大的操作,涉及载荷与应用签名之间的字符串匹配。为解决此问题,Khandait等人提出了一种基于DPI的网络流量分类方法,该方法使用启发式方法,通过单次扫描流载荷即可对网络流进行分类,实现了次线性搜索复杂度[9]。然而,尽管DPI在某些情况下有效且计算开销可能较小,但它可能导致隐私法律问题[10]。此外,由于加密技术和未公开协议的使用,基于DPI的方法可能变得无效。

### 1.2.2 基于传统机器学习的方法

随着机器学习技术的出现, 研究人员主要致力于特征工程。分类准确性的最关键因素变成了如何构造足够有效的特征。基于传统机器学习的方法通常将手动提取的复杂模式或特征应用于监督学习算法作为分类器。

Taylor 等人提出了 AppScanner, 这是一种利用随机森林算法、基于从流数据中提取的统计特征来区分移动应用的分类模型 [11]。Liu 等人提出了一种多属性马尔可夫概率模型, 该模型整合了消息类型序列和数据包长度, 用于对流量应用进行分类 [12]。Ede 等人提出了一种名为 FlowPrint 的移动应用识别方法, 该方法以半监督方式结合了基于目的地的聚类、浏览器隔离和模式识别来进行应用指纹识别 [13]。Taylor 等人探讨了被动窃听者如何通过分析智能手机应用传输的网络流量来识别它们 [14]。作者采用机器学习技术, 基于包括数据包大小和方向在内的侧信道数据来识别智能手机应用, 尽管数据包的有效载荷被 SSL/TLS 加密所隐藏。Shen 等人提出了一种名为 FineWP 的网页指纹识别方法, 该方法涉及累积数据包长度并提取块特征、序列特征和统计特征 [15]。这些特征随后被输入到随机森林、决策树和 k 近邻等机器学习算法中以创建网页指纹。Saber 等人提出了一种主成分分析 (PCA) 和支持向量机相结合的方法用于流量分类, 该方法仅关注基于时间的流特征 [16]。Anderson 等人提出利用逻辑回归算法联合分析流级特征、头部信息以及其他特征, 以将加密流量分类为恶意流量或正常流量类别 [17]。Liu 等人设计了包级统计特征, 包括发送和接收字节的最大值、最小值和平均值, 并提出了一种基于复合特征的半监督方法用于加密流量分类 [18]。Zhang 等人介绍了一种新的鲁棒流量分类 (RTC) 方案, 该方案使用了他们基于词袋 (BoF) 的流量分类器, 并表明其性能显著优于最常见的机器学习方法 [19]。Anderson 等人则使用流级特征, 他们将流元数据、包长度分布、时间分布、字节分布以及未加密的 TLS 头部信息作为联合特征, 利用逻辑回归算法来识别恶意软件加密流量 [20]。Fu 等人将加密流量流以分层方式分割成会话, 并提取包长度和时间延迟序列来构建隐马尔可夫模型 (HMM), 该模型能够识别服务使用情况和终端用户的应用程序内行为 [21]。Feghhi 等人提出仅使用上行链路的数据包时序信息来测量 VPN 流量 [22]。Panchenko 等人首次提出利用累计数据包大小特征生成 Tor 网站指纹, 并被后续研究者广泛采用 [23]。Shen 等人将 CUMUL 特征与上行主导阶段的数据包位置相结合, 构建了一种新颖的细粒度网站指纹识别方法 [24]。

以上基于机器学习的方法都存在一个共性, 其粒度都在流 (flow) 或会话 (session) 级别, 因为要获得通信模式并提取统计特征, 需要观察整个流或会话。这意味着实现早期加密流量分类 (early ETC) 是不可行的, 并且这种方法耗时。同时, 基于机器学习的方法都建立在特征工程的基础上, 这依赖于专家知识。然而, 研

究人员的特征选择工作往往针对特定的加密工具或协议, 这些选定的特征集缺乏泛化能力。因此, 基于机器学习的方法无法快速选择统一的特征来分类加密流量, 不适合流量类型和来源多变的真实网络场景。

### 1.2.3 基于深度学习的方法

受深度学习在图像处理和自然语言处理领域卓越性能的推动, 研究人员将深度学习应用于网络流量分类。基于深度学习的方法不同于传统的基于机器学习的方法或包检测, 因为它们不依赖专家来提取特征。此外, 与传统的机器学习方法相比, 基于深度学习的方法具有更强的学习能力, 因此可以实现更高的性能。

Wang 等人提出了一种用于加密流量分类的端到端方法, 该方法将原始流量的字节数据作为输入, 并将这些数据转换为灰度图像, 再用一维 CNN 进行处理分类 [25]。Liu 等人提出了 FS-Net, 这是一个通过深度神经网络进行加密流量分类的框架 [26]。FS-Net 以数据包长度序列作为输入, 采用双向 GRU 进行特征编码, 并在自动编码器 (AutoEncoder) 中结合了重构机制以保证学习到的特征的有效性。Shapira 等人提出了一种用于加密互联网流量分类和应用识别的新方法, 该方法将基本的流数据转换为直观的图像 (FlowPic), 然后使用卷积神经网络 (CNN) 来识别流类别和正在使用的应用 [27]。Liu 等人提出了一个名为 BGRUA 的模型, 用于识别在 HTTPS 连接上运行的 Web 服务 [28]。BGRUA 采用双向门控循环单元 (GRU) 从会话内的字节序列中捕获前向和后向特征。此外, 还利用注意力机制根据特征对分类任务的贡献程度为其分配权重。Lin 等人提出了一种新的流量表示模型, 称为加密流量双向 Transformer 编码器表示 (ET-BERT) [29]。该模型利用大规模未标记数据预训练深度上下文感知的数据报级表示。他们提出了两项预训练任务: 通过预测被遮蔽的令牌 (masked token) 来捕获流量字节间的上下文关系, 以及通过预测同源突发流 (Same-origin BURST) 来预测正确的传输顺序。Mohammad 等人提出了一种基于深度学习的加密流量分类方法 “Deep Packet” [30]。该方法利用两种深度神经网络结构——堆叠自动编码器 (SAE) 和卷积神经网络 (CNN)——进行应用识别任务。Aceto 等人提出了一种名为 MIMETIC 的多模态深度学习方法。他们分别利用 CNN 和 GRU 来学习两种不同的模态 (有效载荷的前 576 个字节和信息性协议字段), 用于移动加密流量分类 [31]。Wang 等人也提出了一个类似的模型 App-Net, 该模型使用 LSTM 和 CNN 分别学习数据包长度序列和初始数据包的有效载荷字节 [32]。图神经网络 (GNN) 在处理非结构化数据方面具有强大潜力, 并在许多领域展现出良好的性能。Shen 等人提出了 GraphDApp, 该模型能够使用 GNN 识别去中心化应用 (DApps) 的加密流量 [33]。他们基于数据包长度和数据包方向构建了加密流的流量交互图 (Traffic Interaction Graph, TIG), 并将 DApp 识别问题转化为图分类问题。Han 等人提出了

双嵌入图神经网络 (DE-GNN), 分别用 one-hot 编码处理包头和载荷数据, 并设计 PackertCNN 提取包级别特征, 然后构建流量交互图 (TIG) 并使用图神经网络学习流级别特征, 最后用自适应特征融合将二者融合作为最终的模型训练特征用于训练分类 [34]。Yang 等人提出了一个利用双模态特征的 DM-HNN 模型, 该模型同时使用包长度序列作为流级别特征和数据包数据作为包级别特征, 并通过门控激活机制自适应调整特征权重将其融合作为最终用于分类的特征进行模型训练 [35]。

上述方法都取得了不错的成绩, 但是都一定的问题, 大多数方法只考虑了一个模态的特征, 如仅使用流级别或包级别特征, 忽略了另一模态所包含的流量特征。少数多模态方法也仅仅是将两种特征简单拼接, 没有考虑到特征之间的相互关系 [36]。除此之外, 使用流级别特征的方法通常只使用原始包长度序列, 没有考虑到受网络等因素的影响可能产生的序列变化; 使用包级别特征的方法大多数涉及有效载荷数据的使用, 一定程度上损害了用户的隐私权益。因此, 研究一种融合流级别与包级别特征、能够应对包长度序列变化、不依赖有效载荷数据的融合加密流量分类方法具有重要意义。

### 1.3 研究内容

本研究旨在解决当前加密流量分类领域的三个关键问题: 特征维度单一: 现有方法多依赖单一模态特征 (仅使用流级别特征或包级别特征), 忽视双模态互补信息, 导致特征表达不充分; 特征鲁棒性不足: 现有的使用流级别特征的方法通常只使用原始包长度序列, 没有考虑到受网络因素等影响, 可能产生的数据包长度序列变化的问题; 隐私泄露风险: 现有的使用包级别特征的方法, 大多数将包头和有效载荷数据混合使用, 没有对二者的功能和表示进行区分, 一定程度上损害了用户的隐私权益; 融合策略简单: 部分多模态分类方法, 其融合策略 (如直接拼接和加权拼接) 基于固定权重, 没有考虑到不同特征之间的相互关系, 以及各个特征对分类的贡献程度不同的问题。

针对上述问题, 本研究提出以下核心研究内容: 流级别特征鲁棒性提升: 重点解决包长度序列因网络延迟、分片机制产生的结构性失真问题, 构建抗干扰的时序表征模型。包级别特征隐私保护设计: 彻底规避有效载荷数据, 聚焦 IP 包头的的数据, 并在提取过程中保留其结构化的信息 (如协议号、标志位), 探索其与流量类型的隐式关联。双模态贡献度量化: 突破简单特征拼接局限, 建立动态权重分配机制, 在不同类型流量的分类任务中, 根据不同的贡献作用, 自适应地融合两个模态的特征。

1.4 本文主要贡献

1.5 章节安排

## 第2章 相关理论概述

### 2.1 网络流量分类理论

近年来, 由于第五代移动通信技术的快速应用, 各行业对网络需求越来越高, 网络也越来越复杂, 这就造成了网络安全难题的不断增多 [27,28]。网络流量分类技术由于可以有效预防网络攻击, 保证网络质量, 调节网络资源分配, 因此在网络安全和互联网领域发挥着重要作用 [29-31]。网络流量分类是指从网络流量中的数据报文提取出相关特征并对其分类的过程, 网络流量分类的过程大致可以分为三部分, 包括流量数据采集、特征提取、匹配分类 [32,33]。针对网络流量分类问题, 目前已经探索出了许多方法。

#### 2.1.1 端口检测

端口识别是网络流量分类的基础技术之一, 它通过采集网络流量中数据包的端口号, 再进行解析和识别, 就可以确定流量的来源、目的和协议类型等信息。因为每个端口都对应着一个特定的服务或应用程序, 所以通过端口识别就可以初步判断流量的性质。

部分软件通常会使用特定的端口进行通信。因此, 通过对网络流量中的端口号进行识别, 可以初步筛选出特定软件的流量 [34]。此外, 一些恶意软件还会使用非标准端口进行通信, 以逃避检测, 这可以通过对非标准端口进行监测和分析来解决。

基于端口识别的网络流量分类方法主要包括以下步骤: 数据采集、预处理、端口识别、特征提取和分类识别。第一步, 数据采集。主要通过网络监控工具或流量镜像技术获取网络流量数据。在确保数据的完整性和准确性的前提下进行数据采集, 以便后续的分析 and 处理。第二步, 数据预处理, 即对采集到的流量数据进行清洗和整理。由于网络流量通常数据量巨大且格式不一致, 包含大量无效和冗余信息, 因此需要通过数据过滤、去重和格式化等操作, 以便后续的分析 and 处理。第三步, 端口识别。在端口识别过程中, 需要结合已知的端口号和协议信息, 对流量数据进行分类和标注。第四步, 特征提取, 是从流量数据中提取出能够表征网络流量行为的特征, 包括通信频率数据包大小、通信时长, 以及特定的协议字段或行为模式等。通过提取这些特征, 可以构建网络流量的特征向量, 为后续的分类识别提供依据。第五步, 分类识别, 基于提取的特征向量对网络流量进行分类。

基于端口识别的网络流量分类方法目前存在很大局限性。目前, 部分软件会

使用动态端口或加密技术来确保通信的隐蔽性；此外，网络流量的复杂性和多样性也可能对分类识别的准确性产生影响 [35]。

### 2.1.2 深度包检测

基于深度包检测的方法 [8] 也被称为有效载荷分析法，其核心原理是通过对数据包的有效载荷进行深度分析，从而实现对网络流量的分类。该算法的核心在于构建一个网络流量特征数据库，依据不同的应用层协议类型，将各类协议的网络流量特征字段进行分门别类的存储。在实际应用中，深度包检测技术常被用于实时分析并识别流量中的数据包。其主要是通过提取数据包的特征字段，并将这些特征字段与数据库中存储的条目进行逐一比对。通过比对，能够精准判断流量数据包所属的协议类别。此外，还能够进一步对未知应用类型的流量进行精确分类与检测，从而实现对网络流量的精准分类。

与基于端口匹配的网络流量分类方法相比，基于深度包检测的方法由于不受端口号的限制，因此能够更准确地对网络流量进行分类。由于深度包检测方法分析的是数据包中的有效载荷，因此能够识别使用随机端口进行数据传输的应用程序，比如 P2P 应用程序。Sen[9] 等人通过检查可用文档和数据包识别应用程序的签名，最终能够准确地跟踪 P2P 流量。尽管基于深度包检测的网络流量分类方法在分类准确率上优于基于端口匹配的方法，但其也存在一些局限性。首先，由于需要对数据包的有效载荷进行深入分析，处理速度相对较慢，且需要占用大量计算资源和内存，对于大规模网络流量的处理可能带来挑战，同时也难以满足网络流量分类的实时性要求。其次，深度包检测无法对加密流量进行有效的分类和识别，因此在遇到加密流量时可能出现误判或无法进行分类的情况。随着网络技术的迅猛发展，许多应用程序采用负载加密技术，使得该方法的有效性受到了限制。最后，对于新型的应用协议，由于其特征不在特征库中，深度包检测可能无法对其进行准确的分类和识别，随着新型网络应用的不断涌现，该方法很难迅速适应这种网络的高速变化。

### 2.1.3 传统机器学习方法

基于传统机器学习的网络流量分类方法大多使用统计行为作为特征。网络流量在传输过程中表现出不同的统计特征和行为模式，包括数据包的大小分布、传输速度、通信频率、协议类型、端口号等。通过对这些特征进行提取和分析，可以建立网络流量的识别模型，实现对网络流量的有效分类 [37]。

基于统计行为的网络流量分类方法具有高效性。因为该方法无需对流量数据进行逐包解析，所以具有较高的处理效率。同时该方法通过提取丰富的特征信息和行为模式，可以提高分类识别的准确性。并且由于只使用统计特征不使用载

荷内容，该方法可以应用于加密的网络流量分类。

基于统计行为的网络流量分类方法也面临一些挑战。流量数据的统计特征和行为模式对分类性能有影响，不同的特征选择可能导致分类性能的差异，而特征的选择又需要极高的专业知识，且需要丰富的分类经验来选择最为有效的特征。构建高效的分类模型意味着需要大量的已知网络流量数据进行训练，这对样本的数量和质量都有着较高的要求。同时，随着新网络流量类别的不断出现，模型也需要不断更新和优化以适应新的分类任务。

## 2.2 深度学习概述

深度学习 [55] 作为机器学习的关键分支，其核心在于通过建立多层神经网络架构，从数据中自主习得高层次抽象特征。区别于传统机器学习依赖人工特征工程的方式，该方法实现了特征提取过程的自动化。近年来，随着计算资源的增强和算法架构的改进，深度学习技术获得快速发展，已成为处理复杂的模式识别与生成任务的核心方法。

深度学习的核心在于深度神经网络，其通过多层感知机或其他类型的神经网络结构，模拟人脑的信息处理机制。典型的深度学习模型结构由输入层、隐藏层和输出层构成。其中，隐藏层借助非线性激活函数承担特征非线性映射与增强的关键作用。训练时采用反向传播算法调整参数，降低损失值 [56]。深度学习的优势在于其强大的表达能力和灵活性，能够处理海量数据和高维问题。同时，深度学习还衍生出了许多变体，如卷积神经网络 [57]、递归神经网络 [58]、生成对抗网络 [59] 等，这些变体在自然语言处理 [60]、语音识别 [61]、时间序列分析 [62]、自动驾驶 [63] 等特定任务中表现出优秀的性能。伴随计算机相关技术的持续演进和数据规模的不断拓展，深度学习技术在特征提取、模式识别和复杂任务建模等方面仍具备广阔的发展前景 [64]

### 2.2.1 卷积神经网络

卷积神经网络（CNN）是深度学习领域中最成功和流行的模型之一。其广泛应用于计算机视觉、自然语言处理等领域，通过其强大的特征提取和推理能力，在解决复杂问题上表现卓越。CNN 的结构包含多个关键组件，其中包括输入层、卷积层、池化层、全连接层以及输出层，结构如图 2.1 所示。每个组件都发挥着独特的作用，共同构建了网络的功能和性能。具体介绍如下：

#### 1) 卷积层

卷积层在 CNN 中扮演着关键的角色，是用于提取输入数据的局部特征的核心组件。对于一个形状为  $W \times H \times C$  的输入，其中  $W$  和  $H$  分别表示每个输入特

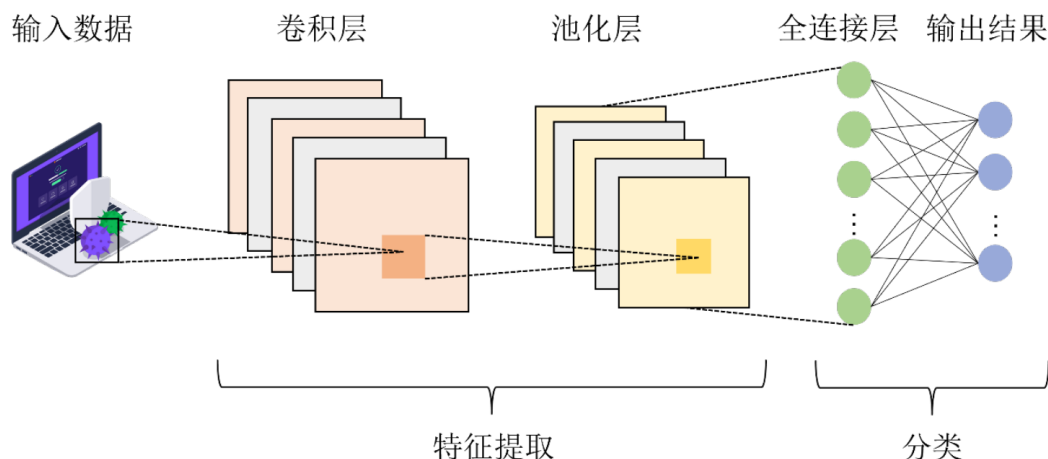


图 2.1 卷积神经网络结构

征图的宽度和高度， $C$  是通道的数量，即特征图的总数。卷积层通过应用一组卷积核对输入特征图执行卷积操作，其中卷积核的数量等于输出特征图的通道数量。每个卷积核均映射于一个特定的输出特征图，不同卷积核在对输入图像进行卷积后产生不同的特征图。

卷积操作涉及多个重要参数，包括输入数据的尺寸  $i$ 、步长  $s$ 、卷积核的大小  $k$ 、输出数据的尺寸  $n$  以及填充大小  $p$ 。步长定义卷积核在滑动时的间隔，填充的选项用于在输入数据的边缘处填充零值，以控制输出数据的空间尺寸。卷积操作后，输出特征图的尺寸可通过公式  $n = (i - k + 2p)/s + 1$  计算。卷积层的卷积核通过学习或预设的方式进行初始化，不同尺寸的卷积核可提取多尺度的特征，赋予模型对输入数据的旋转、平移不变性。重要的是，卷积核的共享权值和非全连接的特性使其能够避免过拟合问题，同时减少网络参数，降低模型的复杂度。

以经典的二维卷积为例，如图 2.2 所示，展示了二维卷积操作的步骤示意图。假设二维卷积的输入为大小为  $4 \times 4$  的矩阵，卷积核大小为  $3 \times 3$ ，卷积步长为 1。经过四次卷积操作后，得到了一个大小为  $2 \times 2$  的输出结果。

## 2) 池化层

池化层是 CNN 中的关键层，通常紧随卷积层之后使用。其主要目的是通过降采样来实现特征的降维，从而有效减少模型训练中的计算负担和参数量，同时能够保留图像中关键的结构信息。此外，池化层还具有在一定程度上抑制过拟合的作用。通过图 2.2 的示意图可见，当特征图的尺寸较大时，卷积核需要滑动的次数增多，相应的卷积运算次数也会显著增加，导致卷积神经网络的计算复杂度提高。因此，为了降低计算负担，引入池化层进行下采样，以降低特征图的维度。同时，池化层的再次运算有助于网络更有效地筛选和提取特征。常见的池化方式主要有最大池化（MaxPooling, MP）和平均池化（Average Pooling, AP）。

图 2.3 展示了不同池化方式的运算过程和结果。假设上层输入为大小为  $4 \times 4$

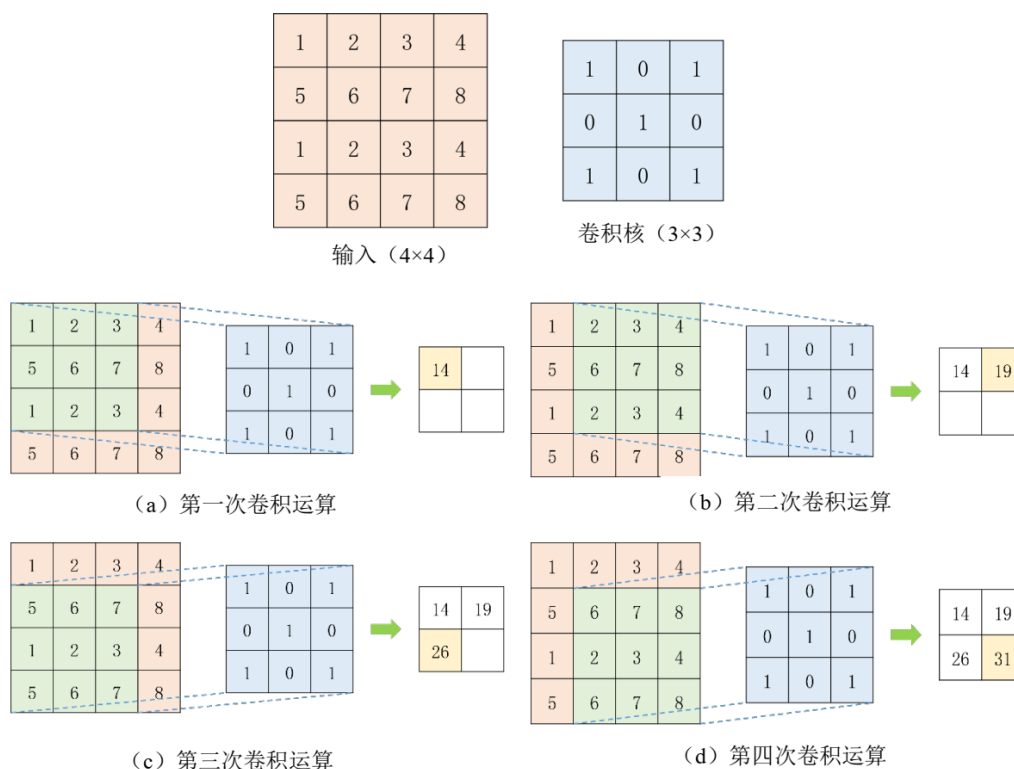


图 2.2 卷积运算过程

的矩阵，池化核大小为  $2 \times 2$ ，池化步长为 2。最大池化与平均池化操作分别对池化核窗口内的数值进行求最大值与取平均值的计算。当池化窗口位于特征图的左上角时，最大池化的结果为池化窗口内的最大值 7，而平均池化的结果为池化窗口内的平均值 4。随着池化窗口在特征图上的移动和计算，可得到相应的池化输出矩阵。在实际应用中，最大池化和平均池化都有广泛的应用场景，需要根据不同的场景选择合适的池化操作，这样才能达到最优的模型性能表现。

3) 全连接层全连接层作为 CNN 的主要构建结构，通常位于网络的最末层。每个神经元在全连接层都与前一层的所有神经元建立连接关系。这一层的核心功能在于整合前层网络提取的各种特征，并将这些特征映射到样本标记空间。线性部分通过不同的角度对输入数据进行全面的分析，形成对输入数据整体的、综合的判断。非线性部分则由激活函数负责，其作用是打破之前的线性映射关系。全连接层的设计在网络的尾部起到关键作用，通过对前层提取的特征进行整合和映射，为网络输出提供最终的预测。每个神经元在全连接层中都承担着对应样本标记空间中的一部分信息，通过连接权重的调整和激活函数的作用，实现对输入数据的复杂非线性映射。这一设计使得网络能够更好地适应复杂的特征关系，从而提高模型的表达能力

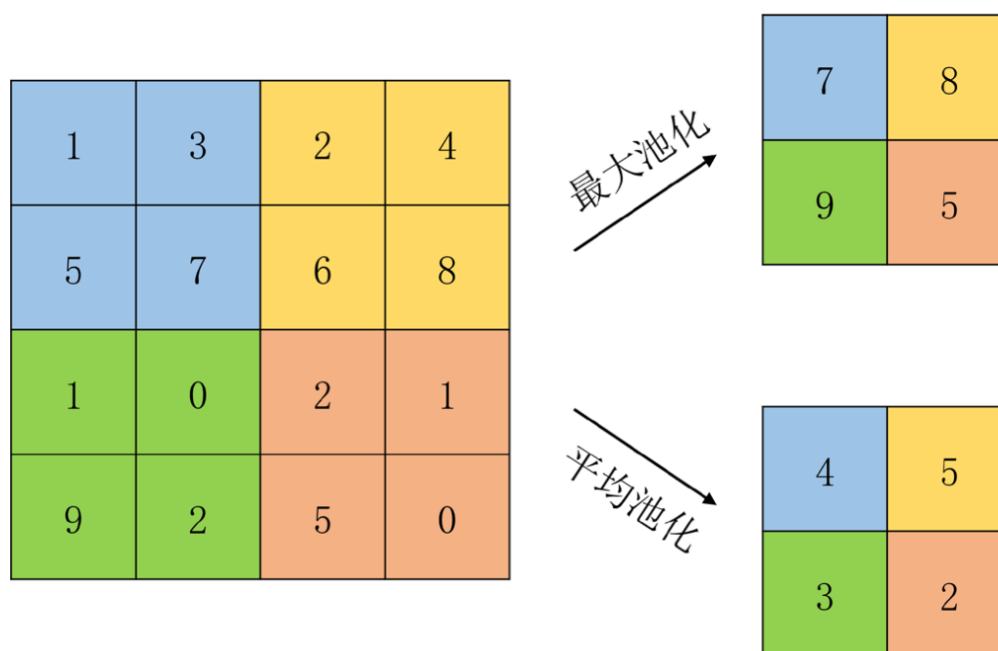


图 2.3 两种池化方式

### 2.2.2 激活函数

深度学习的核心原理建立在人工神经网络的基础上，其中每个神经元接收前一层神经元的输出，并据此生成自身输出，进一步传递至下一层。在多层神经网络的构造中，上层节点的输出与下层节点的输入之间通过一种函数关系相互关联，该函数常被称为激活函数或激励函数，它作为非线性变换的核心要素，在整个模型结构中发挥着至关重要的作用。每一层神经网络都配备有一个激活函数。在神经网络的架构中，如若未对上一层节点的输出施以非线性转换，即便网络层数再深，其性质亦将囿于线性模型的范畴，仅能进行输入的线性组合并产生输出，难以捕捉并学习复杂的映射关系。这使得网络的逼近能力受到限制。因此，通过反复引入非线性激活函数，可以获取更高级别的语义信息，提升深层神经网络的表达能力，使其能够捕捉更为复杂的模式。这一机制使神经网络在多个领域取得卓越成果。

激活函数的选择对深度学习至关重要，它是研究领域备受关注的一部分。随着深度学习技术的不断演进，激活函数的种类也日趋丰富，为神经网络提供了更广泛的选择范围。这些激活函数不仅影响着神经网络的性能，还直接关系到模型在训练过程中的收敛性。常用的激活函数主要包括 Sigmoid 函数、Tanh 函数、ReLU (Rectified Linear Unit) 函数、Leaky ReLU 函数、ELU (Exponential Linear Units) 函数、PReLU (Parametric ReLU) 函数等，图 2.4展示了这些激活函数的函数图像。表 2.1总结了这些激活函数的公式。

Sigmoid 函数是一类广泛采纳的非线性激活函数，其形态呈现为指数函数状，

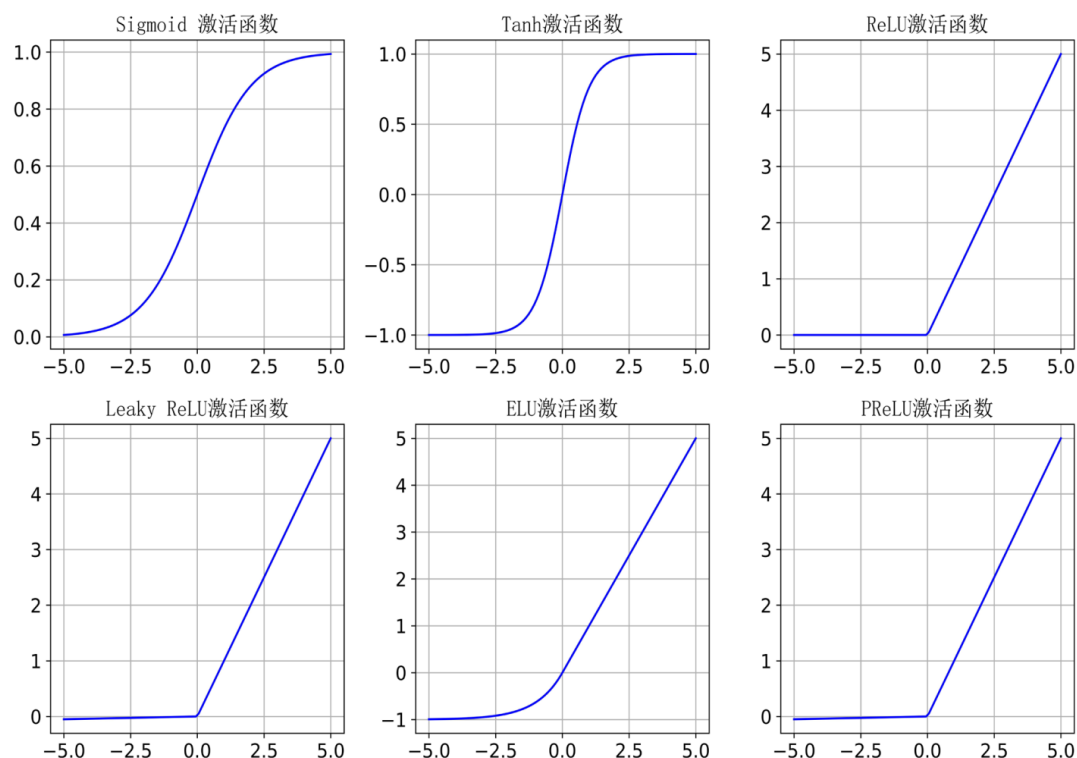


图 2.4 常见的激活函数图像

| 常见的激活函数类型  | $f(x)$                                                                |
|------------|-----------------------------------------------------------------------|
| Sigmoid    | $\frac{1}{1 + e^{-x}}$                                                |
| Tanh       | $\frac{1 - e^{-2x}}{1 + e^{-2x}}$                                     |
| ReLU       | $\max(0, x)$                                                          |
| Leaky ReLU | $\max(0.01x, x)$                                                      |
| ELU        | $\begin{cases} \alpha(e^x - 1), & x \leq 0 \\ x, & x > 0 \end{cases}$ |
| PReLU      | $\begin{cases} \alpha x, & x < 0 \\ x, & x \geq 0 \end{cases}$        |

表 2.1 常见激活函数的公式

从物理学角度来看，其与生物神经元的特性最为吻合。作为神经网络中常见的激活函数，Sigmoid 常被用作神经网络的阈值函数，其功能在于将变量映射至  $[0, 1]$  区间内。然而，Sigmoid 函数存在着梯度消失的问题。其软饱和性质导致对于绝对值较大的数，梯度几乎为零。在深层网络中，若每一层都采用 Sigmoid 函数，多层梯度相乘会导致最终梯度趋近于零，使得参数几乎无法更新，训练无法正常进行，即出现梯度消失问题。

Tanh 是双曲正切函数，也是一种常见的激活函数。从图形上看，Tanh 函数与 Sigmoid 函数具有一定的相似性，然而其输出值范围为  $(-1, 1)$ ，且均值为 0，这有效解决了 Sigmoid 函数输出非零中心的问题。尽管相较于 Sigmoid 函数，Tanh 函数在一定程度上减轻了梯度消失的现象，但其依然存在软饱和性，未能彻底根治梯度消失的问题。

ReLU 函数在深度学习研究中占据重要地位，是一种常见的非线性激活函数，又称修正线性单元。ReLU 函数即取最大值的函数，其导数为 1 或 0。相较于 Sigmoid 和 Tanh 激活函数，ReLU 函数梯度计算简单，显著提升了梯度下降的收敛速度并缓解了梯度消失问题。然而，ReLU 单元相对脆弱，对于小于 0 的输入值，激活后输出和梯度都为零，导致一部分神经元在训练中不可逆地死亡。为解决这一问题，除了标准的 ReLU 函数外，出现了各种 ReLU 函数的变体，统称为 ReLU 函数簇。针对 ReLU 函数的神经元死亡问题，Leaky ReLU 函数通过赋予所有负值一个非零斜率的方式，保留了负轴上的信息，从而解决了 ReLU 函数在负值区域的局限性。

ELU 函数，即指数线性单元，在正值区间输出为本身，在负值区间随着输入趋于负无穷而趋于一个负值  $\alpha$ 。ELU 函数综合了 Sigmoid 和 ReLU 的优点，负轴具有软饱和性，增强了 ELU 对输入变化或噪声的鲁棒性，并通过正值的标识避免了梯度消失问题。同时，ELU 函数的输出值接近于零，加速了梯度下降的速度。

PReLU 函数，即参数化修正线性单元，是带有参数的 ReLU。与常规的 ReLU 不同，PReLU 的负值部分斜率是在训练过程中动态更新的，而非预先设定。当参数  $\alpha$  设置为 0 时，PReLU 将简化为 ReLU；而当  $\alpha$  取值为一个较小的固定数时，PReLU 则近似于 Leaky ReLU。这种灵活的设计使得 PReLU 能够更好地适应不同的数据集和任务需求。

### 2.2.3 损失函数

在深度学习中，损失函数用于度量模型预测值与实际值之间的误差幅度。它为神经网络提供了一个可优化的目标，通过此目标梯度反向传播算法能够进行有效的优化，从而控制神经网络的收敛方向和步伐。损失函数在神经网络中扮演着至关重要的角色，充当了衡量模型学习性能的指标。在深度学习领域，常用的损失函数主要包括均方误差（Mean-square Error, MSE）损失函数、平均绝对误差（Mean Absolute Error, MAE）损失函数、交叉熵（Cross Entropy, CE）损失函数以及 KL-散度（KL-divergence, KL）损失函数等。在这些损失函数中，假设样本数据的真实标签值为  $y_i$ ，模型的预测值为  $p_i$ ，总类别数为  $N$ 。相应的数学公式如表 2.2 所示。

| 常见的损失函数  | 公式                                       |
|----------|------------------------------------------|
| CE 损失函数  | $-\sum_{i=1}^N y_i \log(p_i)$            |
| MSE 损失函数 | $\frac{1}{N} \sum_{i=1}^N (y_i - p_i)^2$ |
| MAE 损失函数 | $\frac{1}{N} \sum_{i=1}^N  y_i - p_i $   |
| KL 损失函数  | $\sum_{i=1}^N y_i \log \frac{y_i}{p_i}$  |

表 2.2 常见损失函数的公式

#### 2.2.4 残差神经网络

残差神经网络（ResNet）是一种由 CNN 改进的创新型神经网络，在图像分类、目标检测等任务具有优越的性能表现，成为深度学习发展历程中的一个重要突破。ResNet 系统性地揭示并分析了深度神经网络中普遍存在的退化问题。传统观点认为，随着网络层数的不断增加，模型应当具备更强的特征提取与表示能力。然而，真实情况却是，当网络深度超越某一阈值后，模型在训练集与测试集上的准确率均出现不升反降的现象。这一现象并非源于过拟合，而是由于网络深度增加所带来的优化难题，具体表现为梯度在反向传播过程中逐渐消失或爆炸，导致深层参数难以有效更新，从而使模型难以收敛至理想状态。为解决上述问题，ResNet 引入了跳跃连接机制，构建了被称为残差块的基本结构单元。在该结构中，输入特征可通过跨层连接直接传递至后方网络层，与经过若干卷积层、批归一化层及激活函数处理后的输出进行逐元素相加。这一设计在数学上实现了对原始映射的残差学习，将原本需要拟合的复杂非线性映射转化为对残差部分的优化，从而大幅降低了深层网络的训练难度。更重要的是，这种结构在反向传播过程中能够构建更为通畅的梯度流动路径，有效缓解了梯度消失问题，使网络在数百甚至上千层的深度下仍能保持稳定的训练动态与高效的特征学习能力。ResNet 的提出不仅在理论上深化了人们对深度神经网络训练机制的理解，也在实践层面极大推动了深度学习模型的发展。它打破了以往对网络深度的限制，使得构建超过数百甚至数千层的超深层网络成为可能，显著提升了模型在各类视觉任务中的表达能力和泛化性能。此外，ResNet 所提出的残差学习思想也被广泛应用于其他网络架构与领域，如自然语言处理、语音识别等，对推动整个深度学习领域的进步产生了深远影响。

### 2.2.5 注意力机制

注意力机制是深度学习的核心建模工具，该机制的研究起源于早期的神经网络模型，最初的目的是为了解决梯度消失或梯度爆炸的问题。随着深度学习理论的发展，该机制在信息筛选与特征增强方面展现出独特的优势，逐渐成为提升模型性能的关键技术。注意力机制的核心思想来源于人类感知系统中的注意力原理，即在处理复杂信息时，人脑会集中注意力在关键部分的特征，而忽略或弱化其他不重要的信息，从而提升模型的性能和效率。该机制使模型能够自动学习重要特征，通过分配不同的权重来增强关键信息的表达，抑制无关信息的干扰。例如，在自然语言处理领域，注意力机制让模型聚焦句子中的关键词，提升语义理解精度。

2014年，Bahdanau 等人 [66] 最早提出该机制并应用于神经机器翻译任务，随后迅速成为深度学习领域的核心研究方向。2017年，Vaswani 等人 [67] 提出的 Transformer 架构通过引入自注意力机制，实现了并行化计算与长程依赖关系的有效建模，利用多头注意力机制建立全局特征关联，显著提升模型性能与训练效率。注意力机制 [68] 依据应用场景与实现架构的差异可划分为若干类型：基于序列内部关联建模的自注意力机制、面向视觉任务的空间注意力机制及通过并行特征子空间协同建模的多头注意力机制。注意力机制的处理过程如图 2.5 所示。

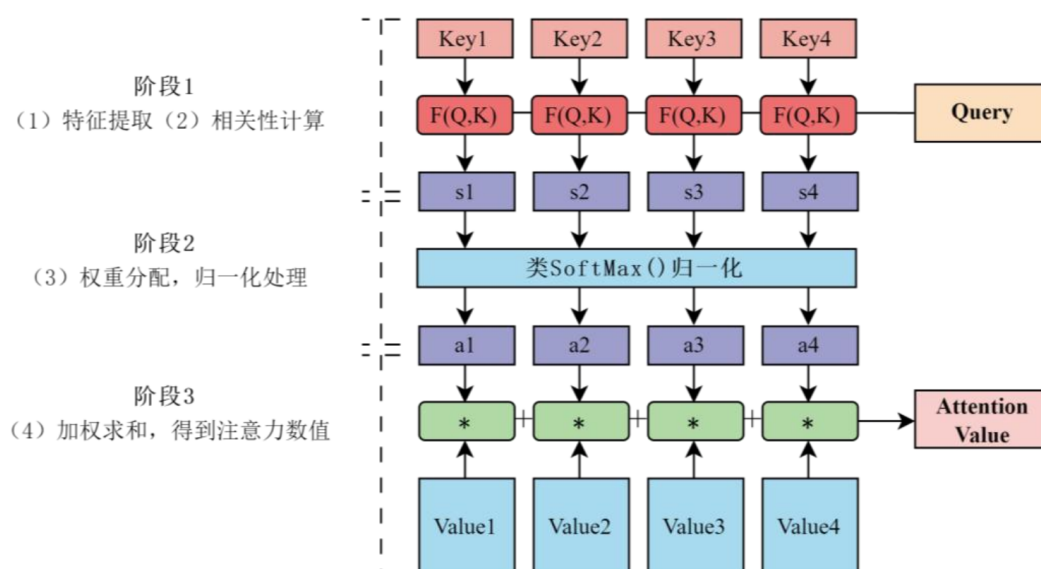


图 2.5 注意力机制处理过程

## 2.3 自监督学习

自监督学习是无监督学习的一个分支，主要解决的是在无法获得标签数据的场景下如何利用未标注数据集中的内在结构信息。自监督学习通过设计的辅助任务机制，能够揭示样本固有的表征特性，并将其转化为模型训练的有效监督信息，从而增进模型的特征提取性能 [59]。自监督学习分为生成式方法与对比式学习方法 [60]。其中，生成式自监督学习主要依托于自编码器架构，例如变分自编码器 VAEs 和生成对抗网络 GANs 等。而对比式学习，则代表了自监督学习中的另一重要方向，相较于生成式方法，对比学习在构建数据深层表征方面表现出更低的任务复杂性及更为显著的优势，已被广泛应用到诸如计算机视觉和自然语言处理等多个领域之中。在网络安全领域，已有研究 [61] [54] 证明对比学习技术具备网络流量分析的能力。

### 2.3.1 对比学习

在自监督学习领域中，对比学习作为一种重要的学习范式，其核心机制体现在设计代理任务和构建目标函数上。这一点与传统有监督学习中直接依赖于真实标签指导模型优化的方式有所区别 [62]。对比学习则是以代理任务为基础，巧妙地定义了正负样本的概念，并通过一个特定的目标函数量化正负样本对之间的损失差异，从而指导模型优化 [63]。

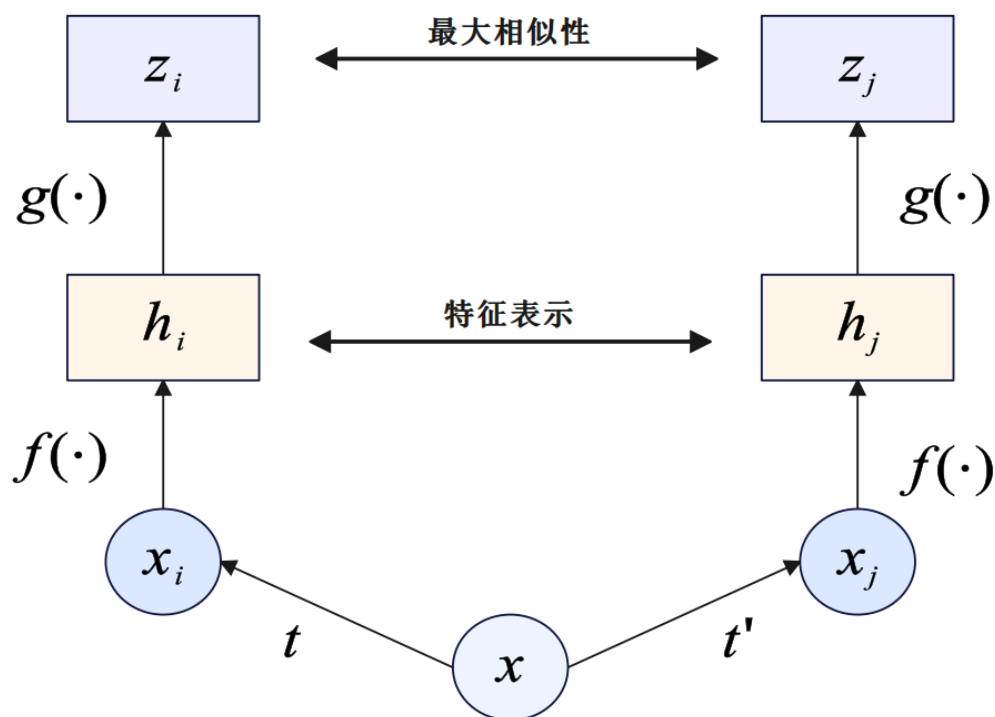


图 2.6 对比学习过程

对比学习的主要目标是实现在嵌入空间中，同一原始样本经不同增强方式产生的新样本之间的距离尽可能接近，而不同原始样本的增强版本之间的距离则尽量拉大。此过程可分为几个步骤，如图 2.6 所示。首先，在代理任务阶段，数据增强是被广泛应用于原始样本  $x$ 。通过应用不同的数据增强方式，样本产生不同视图，进而形成正样本对  $x_i$  和  $x_j$ 。接着，模型引入特征提取编码器  $f$ ，将原始样本的不同视图映射到特征空间，得到特征向量  $h_i$  和  $h_j$ 。之后，这些特征向量通过 MLP 层进行进一步的处理，进一步优化特征表达，得到最终的向量  $z_i$  和  $z_j$ 。最终阶段涉及对比损失函数的设计，其目的是促使模型在特征空间中缩小正样本间的距离，同时增加负样本间的距离。通过这样的方法，模型能够在没有标签的情况下学习区分不同样本，有效提升其在各种任务中的泛化能力。因此，对比学习可以实现在无标签数据上的高效自监督学习，有效提升了模型学习到的特征表达的分度。

### 2.3.2 数据增强

在计算机视觉领域，数据增强技术 [65] 通过引入多种图像变换，增强模型对图像的理解和泛化能力。几何变换，如旋转、缩放、移动和翻转，调整图像的空间布局，帮助模型理解位置偏差。颜色变换通过调整图像的色彩通道和亮度，增加色彩的多样性。旋转和反射变换在特定角度改变图像朝向，增强模型对方向变化的适应性。噪声注入和内核过滤器通过引入视觉扰动，提高模型在不同质量图像上的鲁棒性。混合图像和随机擦除技术通过结合不同图像或随机删除图像区域，强制模型关注图像的多个特征，提升识别能力。这些技术不仅提高了模型的性能，还增强了对复杂场景的处理能力。

在对比学习中，数据增强策略的选择和组合对于提升表示学习的效果有显著影响。对比学习在代理任务阶段通过数据增强技术获得不同的视图，这个过程不仅让模型能够更好地理解数据的核心特质，还大幅提升了其处理未知数据的能力。通过引入各种不同类型的变化，数据增强帮助模型适应这些变化，同时降低过拟合的风险。简而言之，数据增强让对比学习的模型通过训练提升泛化性和鲁棒性，实现更好的效果。Chen 等人 [64] 的研究证明不同的数据增强策略对于模型效果的提升存在显著差异。并且采用多样化的数据增强策略效果优于单一数据增强的效果。因此，设计针对性的数据增强策略是实现高效学习的关键。

## 2.4 本章小结

## 第3章 基于 BYOL 模型的流级特征分类方法

在本章中，我们将详细介绍本文所采用的基于自监督学习架构的流级特征分类模型。该模型核心基于非对称对比学习架构 BYOL，通过在流量数据上进行自编码学习，提取具有强鲁棒性的流级特征表示，再将模型提取后的特征用于下游分类任务。原版的 BYOL 模型处理对象为二维图像，我们对其编码器，增强函数进行了调整，使其能够满足网络流量类型数据的特征提取需求。模型通过使用差异化的增强，构造两种不同的数据包长度序列流量变体，并将同一流量的两个变体输入双网并行架构，进行适当的映射，使二者在高维空间中距离拉近，从而提取出经过增强变化后，两个变体之间共有的，具有鲁棒性的流级别特征。

### 3.1 系统模型

BYOL (Bootstrap Your Own Latent) 是一种基于自监督学习的图像表示学习方法，其核心思想是通过两个神经网络的协同训练实现特征学习，从而为下游任务学习一种高质量的表示方法。该方法的总体架构由两个具有对称结构但更新机制与参数相互独立的神经网络组成：在线网络 (Online Network) 与目标网络 (Target Network)。

在线网络是在线学习的主体，由一组权重  $\theta$  定义，包含三个组成部分：编码器 ( $f_\theta$ )、投影器 ( $g_\theta$ ) 和预测器 ( $q_\theta$ )。它通过梯度下降法来优化参数，学习图像的特征表示。

目标网络在训练中充当在线网络的“导师”，其参数  $\xi$  由另一组权重定义，仅包含编码器 ( $f_\xi$ ) 和投影器 ( $g_\xi$ ) 两个组成部分。目标网络的参数  $\xi$  通过在线网络参数  $\theta$  的指数移动平均值 (EMA) 进行更新。目标网络为在线网络提供稳定的回归目标，同时通过梯度停止机制避免直接参与梯度更新，这种设计使得目标网络能够逐步逼近在线网络的状态。

这种设计使得目标网络能够逐步逼近在线网络的状态，同时避免模型坍塌，从而提升表示学习的鲁棒性。如图 3.1。

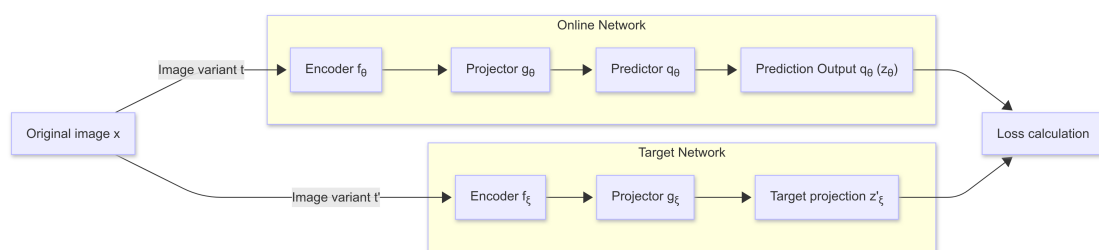


图 3.1 BYOL 模型框架

## 3.2 增强函数

在 BYOL 模型中数据增强操作对于学习良好的数据表征起着至关重要的作用。首先，不同的数据增强操作引入不同的扰动，生成不同增强视角下的样本，模型从中捕捉数据的不变特征，使其在无需负样本的情况下实现高效的特征学习，从而为下游任务提供支持。因此，设计合理的增强函数对于提升模型的性能至关重要。

真实网络环境的复杂性体现在其多变性和不可预测性，尤其是面对网络拥塞、延迟以及带宽波动等问题。如图 3.2 所示，这些网络环境问题可能会导致出现丢包、数据包乱序、数据包重复等问题，拥塞常常会导致数据包丢失，因为路由器或交换机会因为缓存容量不足而被迫丢弃无法及时转发的数据包。其次，为了应对丢包现象，许多传输层协议会启动重传机制，即发送方在一定时间内未收到确认响应时，重新发送已发出的数据包，以保证数据的完整性。另外，原本有序发送的数据包在传输过程中可能因为不同路径的传输延迟差异而到达接收端的顺序发生改变，导致数据包的乱序。Xie 人 [61] 的研究表明目前基于 TCP 可靠传输的数据包长度序列的三个主要变化，即数据包序列移位、数据包序列重复和数据包大小变化。而基于 UDP 的传输方式则还可能出现数据包丢失的情况。通过设计合适的数据增强方式，模拟真实网络环境和加密技术对于流量的影响，可以让模型学习到隐含的本质规则和语义。针对流级别特征提取任务，我们设计了两种增强函数，分别为噪声注入和尺度缩放与特征遮蔽。这两种增强函数能够模拟网络流量数据中的常见变异情况，增强模型的鲁棒性和泛化能力。

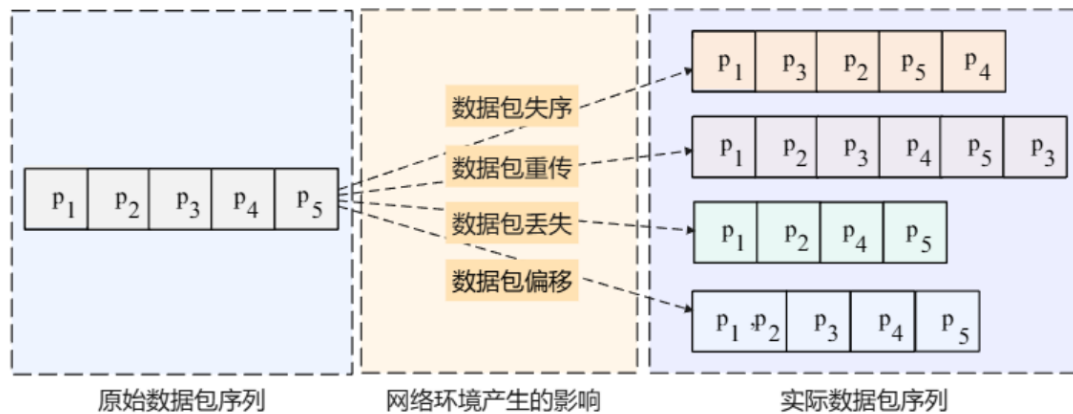


图 3.2 数据包序列变化

### 3.2.1 增强函数 1：噪声注入

第一个增强函数为噪声注入。真实的物理网络中，由于 MTU 限制、中继设备重新分片、协议栈填充或 TCP 粘包拆包等原因，数据包长度并非一成不变，个别数据包可能因为网络条件的变化而出现长度波动。噪声能模拟这种不确定性。

通过向特征向量添加均值为 0、标准差为 0.05 的随机噪声，模型就能够模拟网络测量过程中产生的个别数据包长度变化现象，从而提升模型对网络环境扰动的鲁棒性，本文定义了基于高斯分布的加性噪声变换  $\mathcal{T}_{noise}$ 。对于输入的流级特征序列  $\mathbf{x} \in \mathbb{R}^L$ ，变换后的输出为：

$$\mathcal{T}_{noise}(\mathbf{x}) = \mathbf{x} + \boldsymbol{\eta}, \quad \boldsymbol{\eta} \sim \mathcal{N}(0, \sigma^2)$$

其中， $\sigma = 0.05$ ， $\mathbf{x} \in \mathbb{R}^L$  为原始输入序列， $\boldsymbol{\eta}$  为随机生成的噪声，服从均值为 0、标准差为 0.05 的正态分布。

### 3.2.2 增强函数 2：尺度缩放与随机遮蔽

第二个增强函数为尺度缩放与随机遮蔽。不同层级的隧道协议（如 VPN, TLS, IPv6 额外头部）会对原始载荷添加不同大小的头部，从而使得整个流的数据包长度均发生变化。尺度缩放能让模型学会忽略绝对大小，转而关注包与包之间的相对比例关系。为了模拟这一现象，定义尺度缩放变换  $\mathcal{T}_{scale}$  为：

$$\mathcal{T}_{scale}(\mathbf{x}) = \alpha \cdot \mathbf{x}, \quad \alpha \sim \mathcal{U}(1 - \epsilon, 1 + \epsilon)$$

其中  $\alpha$  是为每个样本独立生成的缩放因子，服从区间为  $[0.8, 1.2]$  的均匀分布（即  $\epsilon = 0.2$ ）。

拥塞的网络环境可能会导致非可靠传输协议传输的数据流发生数据包丢失现象。随机遮蔽则可以模拟数据包在传输过程中丢失或被截断的场景，确保模型在信息不全的情况下依然能做出准确判断。定义掩码变换  $\mathcal{T}_{mask}$ 。首先构造一个二值掩码张量  $\mathbf{M} \in \{0, 1\}^L$ ：

$$\mathbf{M}_{i,j} = \begin{cases} 0, & \text{with probability } P_{mask} \\ 1, & \text{with probability } 1 - P_{mask} \end{cases}$$

随机遮蔽变换为：

$$\mathcal{T}_{mask}(\mathbf{x}) = \mathbf{x} \odot \mathbf{M}$$

其中， $P_{mask}$  是遮蔽概率，设定为 0.1。通过强制移除 10% 的特征信息，迫使在线网络学习利用剩余维度的关联性来重构整体表征，从而提高模型在数据包丢失环境下的判别能力。

通过级联尺度缩放与随机遮蔽，第二个增强函数构建了一个信息更加匮乏的目标视图，使得模型能够深入挖掘特征间的内在关联。

### 3.3 模型架构

#### 3.3.1 编码器

在经过上述增强函数的处理后，模型获得了原始流量数据的两个不同增强变体  $v$  和  $v'$ 。随后，这两个变体被分别输入在线网络和目标网络中的编码器  $f_\theta$  和  $f_\xi$  进行特征表示。在原 BYOL 研究框架中，编码器架构以 ResNet[79] 为基础，用于获取二维图像的特征表示，输入通常是  $(C, H, W)$ ，即（通道、高度、宽度）。而网络流量的数据包长度序列本质上属于一维结构化数据，传统的 ResNet 等图像骨干网并不适用。鉴于此，本文使用一维卷积神经网络作为骨干架构。1D-CNN 的卷积核在单个维度上滑动，这与流量序列按照时间轴线性排列的特征完全吻合，能够通过一维卷积算子在特征轴上有效地捕捉局部统计依赖性，同时避免了二维卷积在处理非图像数据时引入的维度冗余。

本工作设计的编码器采用了逐层特征升维的策略，将初始输入的数据包长度序列，经过多层卷积和池化操作后，最终输出一个 256 维的表征向量。如图 3.3 所示。输入向量为经过标准化的一维向量  $\mathbf{v} \in \mathbb{R}^{B \times 1 \times L}$ ，其中  $B$  为批处理大小， $L$

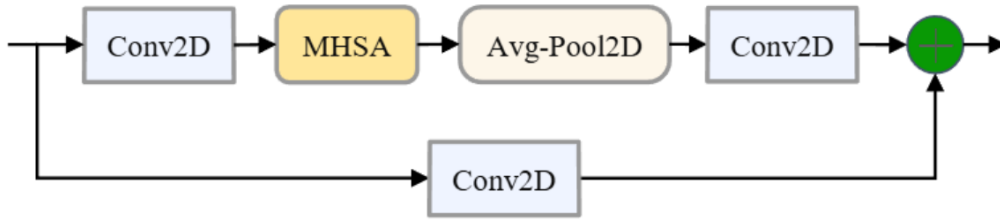


图 3.3 编码器结构

为数据包序列长度。首先，经过浅层特征提取层，使用 128 个大小为 3 的卷积核进行初步扫描，通过 BatchNorm1d 稳定分布并经过 ReLU 进行激活，初步捕捉特征间的局部组合关系。如公式（4）所示：

$$\mathbf{h}_1 = \sigma(\text{BN}(\text{Conv1d}(\mathbf{v})))$$

其中  $\sigma$  代表非线性激活函数 ReLU，BN 代表一维批量归一化。此层将特征维度从 1 提升至 128。

接着，深层语义融合层将通道数分别扩张至 512 和 1024 层，进一步挖掘高阶的非线性空间相关性，这种极宽的网络设计有助于在自监督环境下记录更丰富的特征组合，为下游分类任务留出充足的特征冗余。如公式（5）（6）所示：

$$\mathbf{h}_2 = \sigma(\text{BN}(\text{Conv1d}(\mathbf{h}_1)))$$

$$\mathbf{h}_3 = \sigma(\text{BN}(\text{Conv1d}(\mathbf{h}_2)))$$

然后，经过全局特征聚合层，采用自适应平均池化并展平，保留全局统计特

征,最后通过全连接层将特征投影至 256 维,作为最终的表征向量。如公式 (7) 所示:

$$\mathbf{y} = \text{Pool}(\mathbf{h}_3), \quad \mathbf{y} \in \mathbb{R}^{B \times 1024 \times 1}$$

$$\mathbf{final} = \mathbf{W}\mathbf{y}_{flat} + \mathbf{b}$$

### 3.3.2 投影器和预测器

在自监督学习过程中,若直接在表示空间进行预测可能会导致表征崩溃,即网络发现无论输入什么图像,只要输出一个全零或常数的向量,损失函数就能变成 0,那么它就会停止学习,最终使得在线网络和目标网络的权重和偏置均趋于零,使得所有流量特征都过于近似,无法学习到有效的区分信息。因此,在 BYOL 模型中引入了投影器和预测器两个组件,从而打破了模型的对称性,构建了一个动态系统,迫使模型必须提取有意义的特征才能维持预测的准确性。

首先,通过编码器  $f_\theta$  和  $f_\xi$  对输入的原始流量进行编码后,网络流量被转换为向量形式的特征表示  $\mathbf{y}_\theta$  和  $\mathbf{y}'_\xi$ 。如式 (3.5) 和式 (3.6) 所示,其中  $v$  和  $v'$  是经过两种不同数据增强处理后得到的视图,  $\mathbf{y}_\theta$  和  $\mathbf{y}'_\xi$  分别对应于编码器平均池化层的输出。特征表示  $\mathbf{y} \in \mathbb{R}^d$ , 其中  $d$  是平均池化层输出维度。

$$\mathbf{y}_\theta = f_\theta(v)$$

$$\mathbf{y}'_\xi = f_\xi(v')$$

然后,将得到的特征表示  $\mathbf{y}_\theta$  和  $\mathbf{y}'_\xi$  输入到一个由两个全连接层和一个批量归一化组成的多层感知机中。如式 (3.7) 和 (3.8) 所示,分别对两个网络的向量进行处理,先将特征映射到高维空间,经过批量归一化和非线性激活处理后,再将高维的特征空间投影到低维的特征空间,利用高维中间层作为缓冲,确保特征被压缩为 256 维的投影向量时,仍能保留足够的判别信息,最终得到  $\mathbf{z}_\theta$  和  $\mathbf{z}'_\xi$ 。

$$\mathbf{z}_\theta = g_\theta(\mathbf{y}_\theta) = W_\theta^{(2)} \sigma \left( \text{BN} \left( W_\theta^{(1)} \mathbf{y}_\theta + b_\theta^{(1)} \right) \right) + b_\theta^{(2)}$$

$$\mathbf{z}'_\xi = g_\xi(\mathbf{y}'_\xi) = W_\xi^{(2)} \sigma \left( \text{BN} \left( W_\xi^{(1)} \mathbf{y}'_\xi + b_\xi^{(1)} \right) \right) + b_\xi^{(2)}$$

其中,  $\mathbf{W}$  和  $\mathbf{b}$  分别表示全连接层的权重和偏置, BN 代表批量归一化层,  $\sigma$  是非线性激活函数。特征投影的步骤是至关重要的,不仅能够识别出数据增强中的不变性,还能去除对未知协议流量聚类任务没有意义的信息。通过  $g_\theta$  和  $g_\xi$  的非线性变换,最终  $\mathbf{y}_\theta$  和  $\mathbf{y}'_\xi$  中能够保留更多有用的信息。

预测器  $q_\theta$  是在线网络特有的组件,通过引入额外的非线性映射来打破模型对称性。在特征投影处理之后,初始的特征向量被映射到低维向量空间,产生  $\mathbf{z}_\theta$  和  $\mathbf{z}_\xi$ 。此时,如式 (3.9) 所示,通过在线网络处理的流量需要经过预测器  $q_\theta$  的

处理以获得预测向量  $q_\theta(z_\theta)$ ，而通过目标网络处理的流量则无需此步骤。预测器  $q_\theta$  的网络架构与投影器  $g_\theta$  和  $g_\xi$  保持一致或高度相似，由两个全连接层和一个批量归一化层组成。

$$q_\theta(z_\theta) = W_\theta^{(4)} \sigma \left( \text{BN} \left( W_\theta^{(3)} z_\theta + b_\theta^{(3)} \right) \right) + b_\theta^{(4)}$$

目标网络没有预测器部分，这种结构上的非对称性是 BYOL 模型的核心。如果在线网络和目标网络完全一样，模型通过简单的恒等映射就能使损失函数为零，从而导致表征崩溃。预测器的加入使得在线网络预测目标网络变成了一个非对称任务，这种不对称性迫使模型去学习数据中更复杂的语义，而不是简单的复制。

### 3.3.3 损失函数

在 BYOL 方法中，损失函数是模型优化的核心部分，它通过归一化处理和均方误差衡量在线网络的预测与目标网络的投影之间的差异，从而确保模型稳定收敛。为了增强鲁棒性并避免模型坍塌，BYOL 引入了对称损失，即将两个增强视图进行交换，然后分别计算两次的损失函数并相加，从而得到最终总的对称损失。通过最小化对称损失函数，模型能够在不依赖负样本的情况下学习到高质量的图像表示。

在经过上述投影器和预测器的处理后，两个流量序列增强变体分别通过在线网络和目标网络生成预测向量  $q_\theta(z_\theta)$  和投影向量  $z'_\xi$ 。在线网络预测向量  $q_\theta(z_\theta)$  的维度与目标网络投影向量  $z'_\xi$  相同，确保两者在同一特征空间中进行比较。接着，模型通过最小化预测向量  $q_\theta(z_\theta)$  与目标投影向量  $z'_\xi$  之间的均方误差来对在线网络的参数进行优化训练。

计算损失函数时，首先对  $q_\theta(z_\theta)$  和  $z'_\xi$  进行  $L2$  正则化，公式如下：

$$\bar{q}_\theta(z_\theta) = \frac{q_\theta(z_\theta)}{\|q_\theta(z_\theta)\|_2}, \quad \bar{z}'_\xi = \frac{z'_\xi}{\|z'_\xi\|_2}$$

其中  $\|\cdot\|_2$  表示向量的  $L2$  范数，即向量的欧几里得距离。

然后对两个归一化结果进行均方误差计算，得到初始损失函数，公式如下：

$$\mathcal{L}_{\theta,\xi} = \|\bar{q}_\theta(z_\theta) - \bar{z}'_\xi\|_2^2 = 2 - 2 \cdot \frac{\langle q_\theta(z_\theta), z'_\xi \rangle}{\|q_\theta(z_\theta)\|_2 \cdot \|z'_\xi\|_2}$$

接着需要进行第二次前向传播，交换两个增强视图的输入，将  $v$  和  $v'$  分别输入目标网络和在线网络，得到另一个损失函数  $\tilde{\mathcal{L}}_{\theta,\xi}$ 。最后将两次前向传播的损失函数相加，得到最终的对称损失函数：

$$\mathcal{L}_{\theta,\xi}^{BYOL} = \mathcal{L}_{\theta,\xi} + \tilde{\mathcal{L}}_{\theta,\xi}$$

### 3.3.4 指数移动平均值 (EMA) 更新机制

在线参数更新之后, 还需要对目标网络的参数进行更新, 为了保证学习目标的稳定性, 目标网络的参数  $\xi$  通过在线网络的参数  $\theta$  的指数移动平均进行更新, 在每个训练步骤中, 给定一个预设的衰减率, 目标网络的参数会根据衰减率, 通过在线网络的参数进行加权更新:

$$\xi_t = \tau \cdot \xi_{t-1} + (1 - \tau) \cdot \theta_t$$

其中,  $\xi$  是目标网络的参数,  $\theta$  是在线网络的参数,  $\tau$  为衰减率, 用以控制目标网络参数的更新速度, 衰减率的数值通常非常接近 1, 从而保证目标网络为在线网络提供一个相对稳定, 平滑变化的目标。

这种快慢结合的更新机制一方面为在线网络提供训练动力, 如果两个网络完全一样且同步更新, 那模型容易陷入自己预测自己的死循环, 通过不平衡的更新机制, 在线网络能够不断地学习新的特征信息; 另一方面增强了模型的稳定性, 目标网络的参数更新较慢, 其产生的表示空间不会发生剧烈抖动, 是 BYOL 模型在不使用负样本的情况下依然能避免表示崩溃的关键支撑。

## 3.4 训练优化

### 3.4.1 学习率调度与 AdamW 优化

在本工作的模型优化阶段, 我们采用了 AdamW 优化器。相比于传统的 Adam 算法, AdamW 通过将权重衰减与梯度更新解耦, 能够更有效地提升模型的泛化能力。初始学习率设定为  $2 \times 10^{-5}$ 。为了实现训练全过程的精细化调控, 引入了 ReduceLROnPlateau 学习率调度策略: 当验证集损失在连续 15 个 Epoch 内未取得显著下降时, 系统将触发学习率衰减机制, 以 0.5 的衰减因子对当前学习率进行调整, 旨在通过更小的步长探测损失函数空间的全局极小值点

### 3.4.2 针对抖动的回溯策略

针对自监督学习任务中常见的梯度抖动及局部最优陷阱问题, 本文进一步提出并实现了一套基于参数回溯的重置机制。在实验观察中, 深度模型在处理小样本流式数据集时, 往往因其高度非凸的优化目标而陷入训练僵局。为此, 若验证集损失在连续多个 Epoch 内未能刷新最优记录, 系统将自动判定模型进入收敛瓶颈或过拟合状态。此时, 算法将进行回溯处理。首先, 强制回溯并加载历史保存的最优参数模型, 接着, 对学习率执行阶梯式衰减, 最后, 重置优化器的内部状态, 以消除先前错误梯度的惯性影响。这一策略能够增强模型在复杂训练环境下的鲁棒性, 并有效遏制过拟合现象。

### 3.5 实验设计与结果分析

本节将对所提出的基于 BYOL 的流级别特征分类模型进行全面的实验评估。首先，我们将介绍实验环境的配置，包括硬件和软件环境。接着，详细介绍所使用的数据集及其预处理方法。随后，定义用于评估模型性能的评价指标，并说明模型训练过程中参数设置的细节。最后，与现有的流级别特征分类模型进行对比分析，展示本模型在多个数据集上的表现，并对实验结果进行深入分析。

#### 3.5.1 实验环境

本工作使用的实验硬件设备为 13th Gen Intel(R) Core(TM) i7-13700H (2.40 GHz)，16GB RAM，NVIDIA GeForce RTX 4060 GPU。软件环境方面，操作系统为 Windows 11，使用 Python 3.11 作为主要编程语言。所需要的 Python 库主要包括 scapy，Pytorch，NumPy，Pandas，Scikit-learn 等。实验环境如表 3.1 所示：

| 名称   | 类型                                              |
|------|-------------------------------------------------|
| 处理器  | 13th Gen Intel(R) Core(TM) i7-13700H (2.40 GHz) |
| GPU  | NVIDIA RTX 4060                                 |
| 运行内存 | 16.0GB                                          |
| 操作系统 | Windows 11                                      |
| 软件平台 | Python 3.11                                     |

表 3.1 实验环境设置

#### 3.5.2 数据集介绍与预处理

##### 1) 数据集介绍

为了验证本章所提出的分类方法在加密流量分类任务中的有效性，本工作选取了四个在网络流量识别领域常用的公开数据集作为训练与测试样例。

**ISCX-VPNnonVPN 数据集：**该数据集源自实际网络运行环境，具有较高的真实性和可靠性；包含规模完整的多种协议和环境；包含源 IP、目的 IP、端口、协议、流量大小和时间戳等元数据，方便进行预处理和特征提取；涵盖了通过 VPN 传输的流量和未通过 VPN 传输的流量，且具有多种应用类型，能够满足本文对加密流量特征和行为模式的研究。**ISCX-TorNonTor 数据集：**该数据集侧重于捕获经过洋葱路由器（Tor）多层加密且具备高度混淆特性的匿名流量。涵盖了 Tor 封装与非 Tor 环境下的应用行为，能够检验模型在极端隐私保护及特征强混淆环境下的鲁棒性。**USTC-TFC 数据集：**该数据集根据其原始标签方案将 20 种真实网络流量整合为良性流量与恶意软件流量两大类别，旨在评估模型对非法攻击

及僵尸网络通信的识别精度。**VNAT** 数据集：提供了针对 Skype、Facebook 等十种特定应用程序的细粒度 VPN 流量，并涵盖了关键的命令与控制（command & control, C2）流量场景，为隐蔽通信信道的检测研究提供了数据基础。

在 ISCX VPN-nonVPN 以及 ISCX-TorNonTor 数据集中，原始数据存在标签标注不明确的问题。仅提供了流量文件的描述性信息，而未对其进行精确分类，这导致部分流量文件的类别归属存在歧义，以“Facebook video.pcap”文件为例，该流量文件可能同时符合“浏览器”和“流媒体”两类特征，难以准确归类。为确保实验数据的准确性和模型训练的有效性，本文将容易存在分歧的各类社交媒体流量单独划分为一个新的类别，以区分原有的流量类别，从而形成更为清晰的分类体系。各数据集的具体内容如表 3.2 所示。

| 数据集            | 类型            | 内容                                                                                   |
|----------------|---------------|--------------------------------------------------------------------------------------|
| ISCX-VPNnonVPN | Email         | Email, Gmail                                                                         |
|                | Chat          | ICQ, AIM                                                                             |
|                | Streaming     | Vimeo, Youtube, Netflix, Spotify                                                     |
|                | File transfer | FTPS, SFTP, scp, bittorrent                                                          |
|                | VoIP          | Voipbuster                                                                           |
|                | Socialapp     | Facebook, Hangouts, Skype                                                            |
| ISCX-TornonTor | Email         | Thunderbird, IMAP, POP                                                               |
|                | Chat          | IAM, ICQ                                                                             |
|                | Streaming     | Vimeo, Youtube, Spotify                                                              |
|                | File transfer | P2P, FTP, SFTP, SSL                                                                  |
|                | Web           | Browsing, Google                                                                     |
|                | Socialapp     | Facebook, Hangouts, Skype, Twitter                                                   |
| VNAT           | Streaming     | Vimeo, Netflix, Youtube                                                              |
|                | File transfer | SFTP, RSYNC, SCP                                                                     |
|                | Chat          | Skype                                                                                |
|                | VoIP          | VoIP                                                                                 |
|                | C2            | SSH, RDP                                                                             |
| USTC-TFC       | Benign        | Bittorrent, Facetime, FTP, Gmail, MySQL, Outlook, Skype, SMB, Weibo, WorldOfWarcraft |
|                | Malware       | Cridex, Geodo, Htbot, Miuref, Neris, Nsis-ay, Shifu, Tinba, Virut, Zeus              |

表 3.2 数据集内容

2) 数据预处理在实验执行过程中，为了确保评价结果的客观性与泛化性，本

工作对上述所有原始数据进行了统一的预处理与随机划分：其中 60% 的数据用于模型参数的迭代训练，20% 作为验证集用于超参数调优及防止过拟合，剩余 20% 的数据则作为完全独立的测试集，用于最终衡量模型在未知样本上的分类效能。

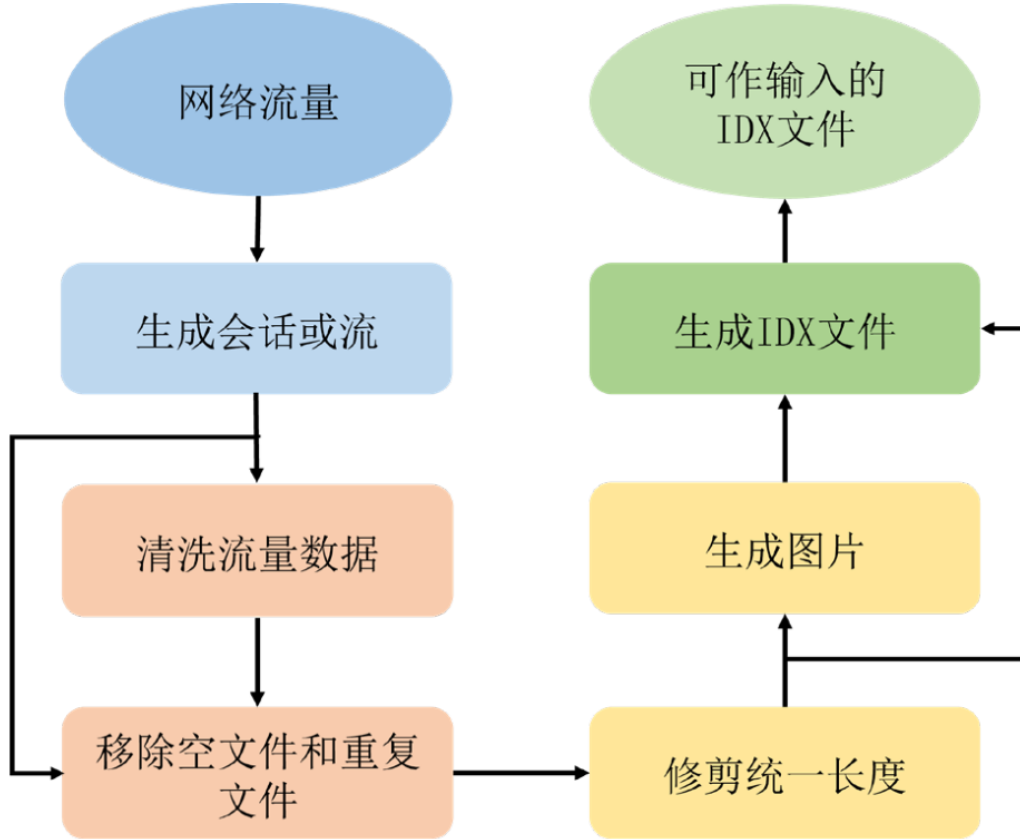


图 3.4 数据预处理流程

在数据预处理阶段，本工作首先利用由源 IP、目的 IP、源端口、目的端口及协议类型构成的五元组作为关联标识，将零散的数据包聚合为具有逻辑意义的网络流。为确保统计特性的鲁棒性并剔除网络扫描或无效连接产生的噪声干扰，本工作对数据包数量少于 5 个的短流进行了过滤剔除。接着，预处理过程从行为模式与内容结构两个维度同步进行多维特征的联合表征：在流级别维度，提取各流前 16 个数据包的应用层负载长度，并对长度不足者实施零填充以规整化为定长向量，旨在有效捕捉加密流量原始的行为序列特征；在包级别维度，则截取每个流的前 5 个数据包并提取其 IP 数据报头，同时剔除源/目的 IP 地址字段，以引导模型学习更具泛化意义的协议结构特征，包级别特征的相关原始数据将为后续章节提出的深度学习方法提供输入支持。最后，针对网络流量数据集中普遍存在的类别不平衡问题，本工作引入了合成少数类过采样技术（SMOTE）与随机欠采样相结合的混合采样策略，通过将各类别样本量统一均衡至各类别数量的中位数，有效缓解了模型对多数类样本的过拟合风险，确保了分类决策边界的公

正性与准确性。预处理过程如图 3.4 所示。

### 3.5.3 评价指标

网络流量识别分类和异常检测，本质上都属于模式识别中的分类问题，这类问题的核心是通过学习流量特征的模式差异，建立从网络数据到类别标签的映射关系。为全面评估分类模型性能，本文采用当前主流的四个评价指标来完成模型的性能评估，分别是准确率（Accuracy, Acc）、精确率（Precision, Pr）、召回率（Recall, Rc）、F1 分数（F1-Score）。同时，为了更直观地展示模型在不同类别上的分类效果。

在多元分类任务中,对于特定类别  $C_i$ ,相关参数定义如下:真正例 (True Positive, TP):实际属于  $C_i$  且被模型正确预测为  $C_i$  的样本数量。假正例 (False Positive, FP):实际不属于  $C_i$  但被模型错误预测为  $C_i$  的样本数量。假负例 (False Negative, FN):实际属于  $C_i$  但被模型错误预测为其他类别的样本数量。真负例 (True Negative, TN):实际不属于  $C_i$  且被模型正确预测为其他类别的样本数量。基于这些定义,上述四个评价指标的计算公式及物理意义如下:

(1) 准确率: 正确分类的样本总数占测试集样本总数的比例,用以评价模型在全数据集上的整体分类效能:

$$\text{Accuracy} = \frac{\sum_{i=1}^n TP_i}{N}$$

其中  $n$  表示类别总数,  $N$  表示测试集样本总数。

(2) 精确率: 在所有被预测为类别  $C_i$  的样本中,实际属于该类别的比例,反映了模型对特定类别预测结果的可靠性。

$$\text{Precision}_i = \frac{TP_i}{TP_i + FP_i}$$

(3) 召回率: 在所有实际属于类别  $C_i$  的样本中,被模型正确识别出的比例,体现了模型对目标类别样本挖掘的完整性。

$$\text{Recall}_i = \frac{TP_i}{TP_i + FN_i}$$

(4) F1 分数: 精确率和召回率的调和平均数,综合考量模型在查准与查全两个维度的平衡表现。

$$\text{F1-Score}_i = 2 \times \frac{\text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}$$

### 3.5.4 参数设置

在本工作中,模型的参数配置旨在平衡自监督学习的表征能力与网络流量数据的计算效率。首先,在骨干网络方面,我们设计了一个深层 1D-CNN 结构,特征通道从输入的 16 维逐步扩张至 1024 维,这种宽通道设计能够有效捕捉流

| 类别        | 参数名称                    | 配置数值                                   |
|-----------|-------------------------|----------------------------------------|
| 训练基础参数    | 批大小 (Batch Size)        | 256                                    |
|           | 最大迭代轮数 (Epochs)         | 500                                    |
|           | 初始学习率 (Learning Rate)   | $2 \times 10^{-5}$                     |
|           | 优化器 (Optimizer)         | AdamW                                  |
| BYOL 核心参数 | 投影维度 (Projection Size)  | 256                                    |
|           | 隐藏层维度 (Hidden Size)     | 4096                                   |
|           | 动量系数 (EMA Decay)        | 0.996                                  |
| 编码器结构     | 输入特征维度                  | 16                                     |
|           | 卷积通道数                   | $128 \rightarrow 512 \rightarrow 1024$ |
|           | 卷积核大小 (Kernel Size)     | 3                                      |
|           | 池化方式                    | AdaptiveAvgPool1d                      |
| 数据增强      | 高斯噪声标准差                 | 0.05                                   |
|           | 随机缩放范围                  | [0.8, 1.2]                             |
|           | 掩码比例 (Masking Ratio)    | 0.1                                    |
| 早停与调度     | 早停耐心值 (Patience)        | 100                                    |
|           | 抖动回溯阈值 (Decay Patience) | 20                                     |

表 3.3 模型及训练参数设置

量序列中微小的统计波动。编码器最终将高维特征压缩为 256 维的向量，作为下游任务的通用表征。在 BYOL 框架配置上，投影器和预测器均采用了多层感知机结构，其中隐藏层维度设定为 4096，这种漏斗形结构有助于在投影空间中进行更复杂的非线性变换，防止表征崩溃。为了维持训练过程中的目标稳定性，目标网络的指数移动平均动量系数设定为模型默认的 0.996，确保目标网络参数能够平滑地演进，从而为在线网络提供一致的优化目标。数据增强策略是模型捕捉不变性的核心。通过加入标准差为 0.05 的高斯噪声、0.8 至 1.2 倍的随机缩放以及 10% 的特征掩码，模型被迫在数据不完整或存在测量误差的情况下提取核心语义特征。最后，在优化策略上，模型采用了 AdamW 优化器，并配合  $2 \times 10^{-5}$  的低学习率以确保微调过程的稳定性。同时，引入了回溯重置机制：当验证集损失连续 20 个 Epoch 无改善时，系统会回滚至历史最佳权重并降低学习率，这种策略结合 100 个 Epoch 的早停机制，能够提升模型在复杂流量数据集上的收敛质量。具体的参数设置如表 3.3 所示。

### 3.5.5 对比算法

为了充分验证本文所提出的基于 BYOL 的流级别特征分类模型在加密流量识别任务中的有效性，我们选择了当前领域内具有代表性的几种流级别特征分类算法作为对比对象。这些算法涵盖了传统机器学习方法、基于深度学习的方法以及对模型进行消融处理的方法，能够全面展示本模型在不同类型算法中的性能优势。具体的对比算法包括：

(1) 支持向量机 (SVM)：作为传统的机器学习方法，SVM 在流量分类任务中以其良好的泛化能力和对高维数据的适应性而被广泛应用。我们将使用原始数据包长度序列来训练 SVM 模型，以评估其在加密流量分类任务中的表现。

(2) FS-Net：FS-Net 是在深度学习领域经典的端到端加密流量分类框架，且在流量分类领域是最常用的对比模型之一。采用了基于多层双向门控循环单元 (bi-GRU) 的编码器-解码器架构，其核心设计思想在于引入了重构机制。在该框架下，模型不仅通过有监督学习进行应用分类，还通过无监督的重构任务，要求解码器根据编码器提取的特征还原原始流量序列，从而增强特征表示的判别能力与鲁棒性。我们使用每个数据流的前 100 个数据包长度序列作为输入特征，训练 FS-Net 模型。

(3) BYOL-Base：为了验证本文所提出的增强函数和模型架构设计对性能提升的贡献，我们还将设计了一个基于 BYOL 的基础版本作为对比算法。该版本将仅使用原始数据包长度序列进行训练，并且不包含任何数据增强操作，同时保持编码器、投影器和预测器的结构不变。通过与 BYOL-Base 的对比，我们能够量化增强函数在提升模型性能方面的作用。

### 3.5.6 实验结果分析

在本节中，我们将对所提出的基于 BYOL 的流级别特征分类模型与上述对比算法在多个数据集上的实验结果进行详细分析。首先，我们将展示各算法在测试集上的准确率、精确率、召回率和 F1 分数等评价指标的具体数值，并通过表格和图形的形式进行比较。四个数据集的实验结果如表 3.4 所示，其中 Precision, Recall, F1-Score 的值为每一个类别对应值的平均数。

从整体实验数据来看，本章提出的方法 BYOL-Ours 在 ISCX-VPNnonVPN、ISCX-TornonTor、USTC-TFC 和 VNAT 四个主流数据集上均取得了最优的分类表现。在各项核心评估指标中，本模型不仅显著超越了基于传统机器学习的 SVM 算法，且相较于不经过增强处理的 BYOL-Base 框架在准确率上实现了大幅度跃升，平均提升幅度接近 12% – 18%。这有力地证明了针对加密流量特性所做的架构改良与增强函数优化，能够让模型更精准地捕捉报文流中的时空关联特征。与在流量分类领域广泛使用的 FS-Net 方法相比，本章提出的方法依然在所有测试

| 数据集            | 方法               | Accuracy      | Precision     | Recall        | F1-Score      |
|----------------|------------------|---------------|---------------|---------------|---------------|
| ISCX-VPNnonVPN | SVM              | 65.2%         | 64.1%         | 63.9%         | 64.0%         |
|                | FS-Net           | 86.20%        | 86.70%        | 86.14%        | 86.14%        |
|                | BYOL-Base        | 79.1%         | 78.9%         | 79.0%         | 79.0%         |
|                | <b>BYOL-Ours</b> | <b>88.41%</b> | <b>88.66%</b> | <b>88.38%</b> | <b>88.32%</b> |
| ISCX-TornonTor | SVM              | 65.2%         | 64.1%         | 63.9%         | 64.0%         |
|                | FS-Net           | 87.02%        | 87.08%        | 87.01%        | 86.97%        |
|                | BYOL-Base        | 79.1%         | 78.9%         | 79.0%         | 79.0%         |
|                | <b>BYOL-Ours</b> | <b>92.03%</b> | <b>91.94%</b> | <b>91.99%</b> | <b>91.94%</b> |
| USTC-TFC       | SVM              | 65.2%         | 64.1%         | 63.9%         | 64.0%         |
|                | FS-Net           | 92.67%        | 93.45%        | 92.51%        | 92.62%        |
|                | BYOL-Base        | 79.1%         | 78.9%         | 79.0%         | 79.0%         |
|                | <b>BYOL-Ours</b> | <b>97.81%</b> | <b>97.85%</b> | <b>97.82%</b> | <b>97.81%</b> |
| VNAT           | SVM              | 65.2%         | 64.1%         | 63.9%         | 64.0%         |
|                | FS-Net           | 93.40%        | 93.19%        | 92.62%        | 92.77%        |
|                | BYOL-Base        | 79.1%         | 78.9%         | 79.0%         | 79.0%         |
|                | <b>BYOL-Ours</b> | <b>94.01%</b> | <b>93.87%</b> | <b>93.95%</b> | <b>93.88%</b> |

表 3.4 不同数据集上的对比实验结果

场景中保持领先，尤其在处理高度混淆的匿名流量与隧道流量时，展现出了更强的判别力与稳定性。

在 ISCX-VPNnonVPN 与 ISCX-TornonTor 这两类高难度数据集上，本章提出的方法表现尤为突出。在 VPN 隧道识别任务中，由于流量经过二次封装导致特征高度平滑，传统方法如 SVM 仅能达到 65.2% 的准确率，而本章提出的方法成功将其准确率提升至 88.41%。类似地，在面临具有多层加密与高度匿名特性的 Tor 流量时，本模型取得了 92.03% 的准确率，相较于 FS-Net 领先了约 5%。这说明本算法通过优化的对比学习机制，能够有效过滤加密层带来的噪声干扰，从看似随机的数据分布中提取出具有高度区分度的底层协议表征，从而在复杂多变的加密隧道分类中保持高水准的分类性能。

在安全监测与实际应用环境的验证中，模型在 USTC-TFC 与 VNAT 数据集上的表现进一步证明了其工程应用价值。在涵盖多种恶意流量的 USTC-TFC 数据集上，本章提出的方法取得了最好的性能表现，极高的精确率与召回率意味着模型在恶意流量检测任务中具有极低的误报与漏报概率，这对于构建高效的网络防御体系至关重要。而在模拟真实多样化应用场景的 VNAT 数据集中，与对比模型 FS-Net 相比，本章提出的方法通过更健壮的流级别特征提取的方式实现了

分类性能的跨越，准确率达到 94.01%。综合以上分析，本研究所提模型在分类精度与泛化能力上均达到了先进水平，能够有效应对加密网络流量中的分类问题。

### 3.6 本章小结

## 第 4 章 基于融合特征的分类方法

在上一章中，我们介绍了基于 BYOL 模型的流级特征分类方法，该方法利用数据包长度序列作为流级别特征的原始表示，在处理加密网络流量时展现出了良好的分类性能。然而，仅依赖单一维度的流级别特征在应对复杂的真实网络环境时，往往难以全面且细致地捕捉流量模式的深层异构性。

图 4.1 为 VNAT 数据集中，提取前 16 个数据包和前 100 个数据包作为原始流量的分类结果混淆矩阵，可以看出：在仅提取 16 个数据包长度序列的短序列情况下，VNAT 数据集中的 C2 和 file 类别存在严重的混淆情况，近 50% 的 C2 类别流量被误分类为 file 类别。而将数据包数量提升至 100，提取长序列以后，C2 被误分类的情况得到缓解，但却引发了过矫正问题，超过 10% 的 file 流量误分类为 C2 类别。这种分类性能的震荡现象表明，这两种流量在数据包长度序列上的特征表示具有极高的相似度。仅凭流级特征，模型难以精准区分两者在行为模式上的细微差别，从而导致分类边界模糊，陷入分类性能的瓶颈。

针对上述挑战，本章提出了一种双模态特征融合的分类架构。首先，为了弥补流级别特征在微观表现力上的不足，我们引入了基于 n-gram 技术的包级别特征提取方法，旨在从数据包内容中挖掘更具辨别力的模式特征。其次，考虑到流级特征与包级特征在信息贡献度上的差异性，本章进一步设计了一种基于注意力机制的特征融合方案。该方案能够动态地捕捉并加权不同尺度的关键信息，通过流级别与包级别特征的深度互补，构建出一个鲁棒性更强、识别精度更高的加密流量分类模型，旨在从根本上缓解复杂流量类别间的混淆问题。

| Test Classification Report: |              |           |        |          |         | Test Classification Report: |              |           |        |          |         |
|-----------------------------|--------------|-----------|--------|----------|---------|-----------------------------|--------------|-----------|--------|----------|---------|
|                             |              | precision | recall | f1-score | support |                             |              | precision | recall | f1-score | support |
|                             | chat         | 1.0000    | 1.0000 | 1.0000   | 194     |                             | chat         | 0.9510    | 1.0000 | 0.9749   | 194     |
|                             | file         | 0.6771    | 0.9949 | 0.8058   | 196     |                             | file         | 0.9058    | 0.8827 | 0.8941   | 196     |
|                             | streaming    | 0.9862    | 0.9817 | 0.9839   | 218     |                             | streaming    | 0.9855    | 0.9358 | 0.9600   | 218     |
|                             | VoIP         | 0.9902    | 1.0000 | 0.9951   | 202     |                             | VoIP         | 0.9902    | 1.0000 | 0.9951   | 202     |
|                             | C2           | 0.9897    | 0.5053 | 0.6690   | 190     |                             | C2           | 0.8608    | 0.8789 | 0.8698   | 190     |
|                             | accuracy     |           |        | 0.9010   | 1000    |                             | accuracy     |           |        | 0.9400   | 1000    |
|                             | macro avg    | 0.9286    | 0.8964 | 0.8908   | 1000    |                             | macro avg    | 0.9387    | 0.9395 | 0.9388   | 1000    |
|                             | weighted avg | 0.9298    | 0.9010 | 0.8945   | 1000    |                             | weighted avg | 0.9404    | 0.9400 | 0.9399   | 1000    |

| Normalized Confusion Matrix (row-wise): |        |       |           |        |       | Normalized Confusion Matrix (row-wise): |        |       |           |        |       |
|-----------------------------------------|--------|-------|-----------|--------|-------|-----------------------------------------|--------|-------|-----------|--------|-------|
|                                         | chat   | file  | streaming | VoIP   | C2    |                                         | chat   | file  | streaming | VoIP   | C2    |
| chat                                    | 100.0% | 0.0%  | 0.0%      | 0.0%   | 0.0%  | chat                                    | 100.0% | 0.0%  | 0.0%      | 0.0%   | 0.0%  |
| file                                    | 0.0%   | 99.5% | 0.5%      | 0.0%   | 0.0%  | file                                    | 0.0%   | 88.3% | 0.0%      | 0.0%   | 11.7% |
| streaming                               | 0.0%   | 0.5%  | 98.2%     | 0.9%   | 0.5%  | streaming                               | 3.7%   | 0.0%  | 93.6%     | 0.9%   | 1.8%  |
| VoIP                                    | 0.0%   | 0.0%  | 0.0%      | 100.0% | 0.0%  | VoIP                                    | 0.0%   | 0.0%  | 0.0%      | 100.0% | 0.0%  |
| C2                                      | 0.0%   | 48.4% | 1.1%      | 0.0%   | 50.5% | C2                                      | 1.1%   | 9.5%  | 1.6%      | 0.0%   | 87.9% |

图 4.1 两个混淆矩阵

## 4.1 系统模型

本章提出了一种基于双模态融合特征的深度学习网络流量分类模型。该模型旨在解决单一维度特征在面对复杂加密流量时表达能力不足的问题。系统整体架构由三个核心阶段组成。首先是包级别特征的增强型 N-gram 序列建模，用于挖掘报文内部的字节结构信息和文本顺序信息；其次是基于交叉注意力机制的特征融合模块，将流级别特征与包级别特征进行非线性对齐与融合，利用交叉注意力机制挖掘两种维度特征间的相关性，并在训练过程中动态地调节两种特征的权重从而达到更好的融合效果。最后，我们还设计了深度残差分类器 MLP 分类器用于进行分类任务，分类器主体采用了由多个残差块堆叠形成的网络结构，每个残差块内部均包含全连接层、批归一化与 Dropout 结构，通过引入跳跃连接机制，有效缓解了深层网络中的梯度消失问题，并提升了模型的表达能力和泛化性能，确保模型能够精准捕获复杂的流量分类边界并最终输出准确的流量类别。

## 4.2 包级别特征提取

在传统的基于数据包内容的分类方法中，通常依赖数据包的有效载荷作为核心识别特征。然而，有效载荷中往往包含用户身份、通信内容及行为偏好等高度敏感的信息，直接将其作为深度学习模型的输入，极易引发潜在的隐私泄露风险与伦理争议。此外，随着各种加密协议的普及，有效载荷呈现高度熵增的伪随机状态，导致传统的载荷分析手段较难达到准确的分类效果。

Hu 等人的研究发现，虽然加密协议屏蔽了应用层内容，但网络层与传输层的数据包首部数据仍保留了丰富的协议交互逻辑与特性。其实验结果表明，仅通过提取报文头部的字节序列进行建模，即可在多种复杂业务场景下实现与全载荷方案相媲美的分类精度。基于此，本工作在确保用户隐私边界的前提下，将有效载荷数据提出，仅使用 IP 数据报头部数据作为模型的原始输入。通过对报文头部进行结构化特征挖掘，旨在构建一种既符合隐私保护要求，又能适配加密网络环境的分类模型。

### 4.2.1 报文首部截取与预处理

为了捕获网络协议交互的核心语义并构建特征输入，我们首先对原始流量进行流分割，基于五元组信息源 IP、目的 IP、源端口、目的端口及传输层协议将捕获的数据包切分为逻辑连接的数据流。接着预先截取每个流前 5 个数据包的 IP 报文首部作为原始特征输入。

在特征空间构建过程中，需针对报文首部的结构特性进行处理。标准的 IPV4

首部包含 20 字节的固定长度字段，其结构如表 4.1 所示。IP 分组报头涵盖了段版本、首部长度、优先级与服务类型、总长度、标识、标志、段偏移、生存时间、协议、校验和、源 IP 地址和目的 IP 地址这 12 个关键字段。然而，源/目的 IP 地址字段与特定的网络拓扑配置高度耦合，其数值分布更多反映了网络部署环境而非业务流本身的协议特征。从网络流量分类任务的角度来看，这两个字段属于冗余信息，会影响模型的泛化能力和分类结果。因此，本工作在预处理阶段将源/目的 IP 地址字段视为剔除，从而实现特征降维与关键语义的聚焦。

| 0-3 bit  | 4-7 bit | 8-15 bit | 16-18 bit | 19-31 bit |
|----------|---------|----------|-----------|-----------|
| 版本       | 首部长度    | 服务类型     | 总长度       |           |
| 标识       |         |          | 标志        | 片偏移       |
| 生存时间     | 协议      | 首部校验和    |           |           |
| 源 IP 地址  |         |          |           |           |
| 目的 IP 地址 |         |          |           |           |
| 可选字段     |         |          |           | 填充        |

表 4.1 IP 数据报头结构

#### 4.2.2 N-gram 嵌入处理

在现有的基于数据包内容的分类方法中，主流方法通常将报文视为独立的字节流，并将每个字节视为离散的输入特征。然而，IP 数据包的字节序列具有比自然语言更为严谨的结构化语义。报文首部中的多个连续字节通常共同构成一个具有明确物理含义的字段，例如总长度字段由两个字节组成，段偏移字段涉及位运算关联。此外，报文首部字节间存在固定的位置关系，这些结构化信息与顺序特征对于识别协议模式至关重要。融入这些信息可以增强特征表示。为了充分挖掘字节间的结构化信息与顺序特征，本工作引入了 N-gram 嵌入方案。通过在预处理后的报文首部滑动取窗，提取具备上下文关联的语义特征。

本工作使用大小为  $n \in \{1, 2, 3\}$  的滑动窗口对报头数据进行扫描。将 1 字节、2 字节和 3 字节的段作为特征生成的标记，所有窗口的步长均为 1。对于预处理过程中获得的 IP 数据包报头，1-gram 捕捉孤立的字节信息，而 2-gram 和 3-gram 通过构建长度分别为 2 和 3 的字节组合，提取报文内部的结构化信息与顺序特征。最后将得到的嵌入式特征按顺序连接起来，形成最终的包级特征。整个过程如图 3 所示。

对于 1-gram 得到的字节形式特征，我们对其字节流进行归一化处理。以此

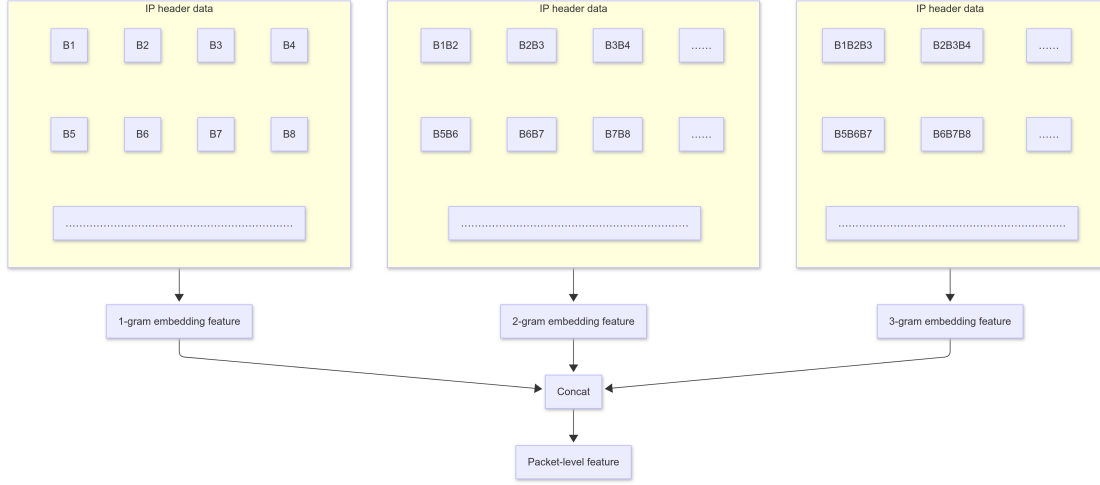


图 4.2 n-gram 嵌入处理

消除量纲差异并加速深度学习模型的梯度收敛。本工作将原始字节在  $[0, 255]$  范围内的取值线性映射至  $[-1, 1]$  的数值区间。这种处理方式在保持报文首部关键字段原始分布特征的同时，通过中心化与缩放操作提升了线性特征的分布稳定性，为后续通过非线性模型提取包级别特征奠定基础。

而对于 2-gram 和 3-gram 得到的字节组合特征，我们将其十六进制字节组合转换为对应的十进制数值  $ngram_{dec}$ 。通过这种映射，原始的离散字节被转换至具备数值统计意义的高维特征空间中。由于网络协议中存在诸如标识符、校验和或高精度时间戳等字段，其产生的  $ngram_{dec}$  往往呈现极高的随机性以及长尾分布。若采用传统的线性归一化方法，由于 n-gram 理论最大值  $16^{2n} - 1$  极大，会导致大部分关键信息被压缩至极小的数值区间内，产生严重的特征淹没效应。因此，我们引入了非线性对数变换来缓解这一问题。

$$\log\_val = \ln(1 + ngram_{dec})$$

通过对  $ngram_{dec}$  进行对数变换，模型能够在保持原始数值单调性的前提下，有效地压缩高值区间、扩张低值区间，同时保留特征之间的相对关系，从而缓解数据分布极度不均的问题。随后，利用理论最大值的对数映射  $\log\_max = \ln(1 + \max\_val)$  对数据进行重缩放，将其映射至与 1-gram 特征相同的  $[-1, 1]$  区间内，确保不同 n-gram 特征在同一数值范围内进行融合与学习。

$$normalized = 2 \times \frac{\log\_val}{\log\_max} - 1$$

这种处理方式不仅保留了数据报头之间的词组信息和各关键字之间的顺序信息，还有效地压缩了极端值的影响，使得模型能够更好地学习到数据包级特征中的细微差异，从而提升分类性能。

为了使后续下游分类任务能够正常进行，我们还将处理后的特征进行进行维度对齐，从而保证分类模型输入张量的维度一致性。针对通信持续时间较短、

数据包总数不足5个的短流，模型通过计算现有长度来动态生成占位包。填充数值统一设定为-1，以区分于正常数据包的特征值。这种填充方式不仅保证了多级特征按顺序连接后的特征向量长度一致，还通过显式的边界填充符号降低了填充数据对模型真实特征学习的干扰，帮助模型在训练过程中识别并适应不同长度流量的特征分布差异，从而提升模型的泛化能力和分类性能。

### 4.3 交叉注意力特征融合机制

在完成上述包级别特征与上一章流级别特征的提取后，我们还需要将两个模态的两类特征进行高效地融合，从而提升模型的分类性能。传统的特征融合方法多采用简单的向量拼接或加权求和，这种线性处理方式往往忽略了流级别特征与包级别特征之间深层的非线性关联，难以充分发掘不同模态特征在刻画特定网络行为时的互补潜力。为了深度挖掘这种关联信息，本工作构造了基于交叉注意力机制的特征融合模块。该模块通过模拟人类视觉注意力的分配逻辑，以流级别特征作为引导信号，动态地在包级别特征序列中检索并聚焦于关键的报文模式。通过在隐空间内进行特征重构与空间对齐，该机制不仅能够自动学习不同模态特征的贡献权重，还能有效缓解异构数据间的维度冲突，从而生成具备高度判别力的融合特征，为后续分类网络提供更加有助于流量分类的输入。

#### 4.3.1 线性投影与空间对齐

由于流级别特征  $F$  侧重于刻画通信过程的时间序列信息与包级别特征  $P$  侧重于刻画数据包的结构化信息，两类特征在特征维度、取值范围以及语义空间上存在显著差异，直接进行向量拼接或加权融合会导致维度失配，并可能因特征量级不一而引发特征淹没，进而造成关键信息的丢失。为此，本模块首先引入了维度对齐机制，通过学习两个独立的线性全连接层，将异构的流级别特征空间与包级别特征空间映射至一个统一的、高维的隐空间中。从而实现维度的强制对齐，并对两类特征进行重新编码，使其在同一语义空间内进行交互与融合。

$$f_{align} = \text{LayerNorm}(W_f \cdot F + b_f)$$

$$p_{align} = \text{LayerNorm}(W_p \cdot P + b_p)$$

其中， $W_f \in \mathbb{R}^{d_{align} \times d_f}$  和  $W_p \in \mathbb{R}^{d_{align} \times d_p}$  为可学习的投影权重矩阵， $b_f$  与  $b_p$  为对应的偏置项， $d_{align}$  表示对齐后的空间维度。

#### 4.3.2 交叉注意力流

在完成空间对齐后，本工作利用交叉注意力机制实现流级别特征与包级别特征之间的深度交互。如图4.3所示，与传统自注意力机制中  $Q$ 、 $K$ 、 $V$  均来自同

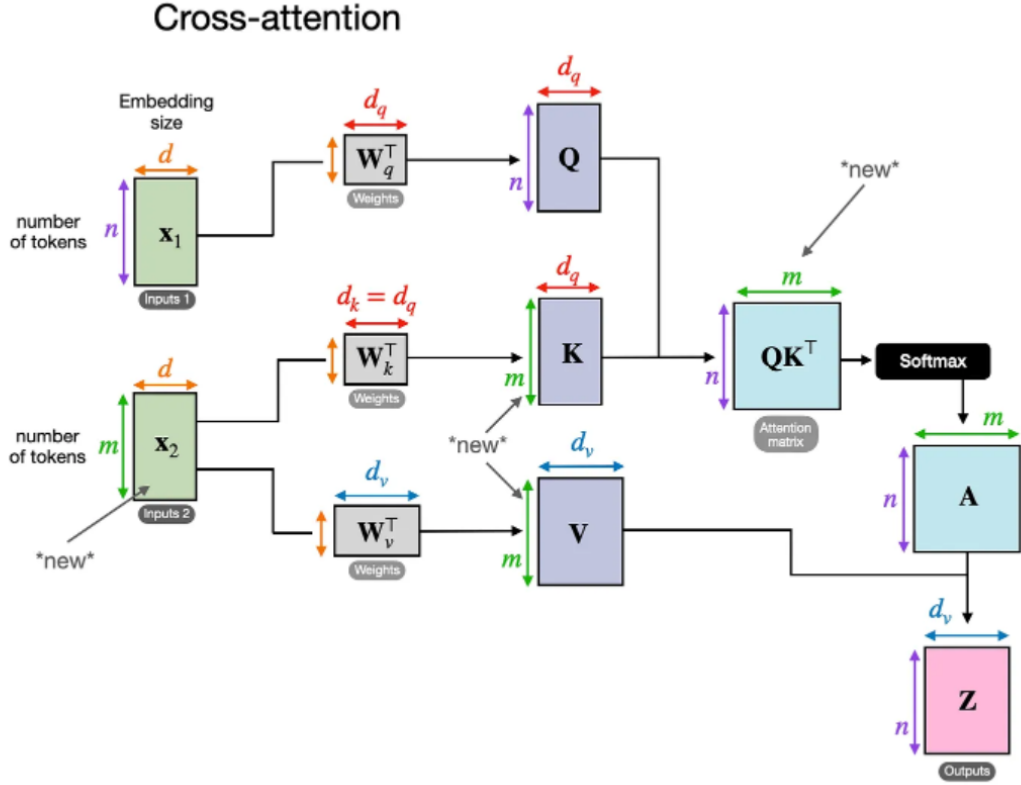


图 4.3 交叉注意力机制

一模态的逻辑不同，交叉注意力机制旨在建立不同模态特征间的协同映射关系。在本工作中，我们利用流级特征  $f_{align}$  进入注意力流生成查询向量 Query，主动检索包级特征  $p_{align}$  生成的键向量 Key 和值向量 Value。利用较为宏观的数据包长度序列信息去询问相对局部的数据包报文信息。具体公式如下：

$$Q = f_{align}W^Q + b^Q$$

$$K = p_{align}W^K + b^K$$

$$V = p_{align}W^V + b^V$$

其中， $W^Q, W^K, W^V \in \mathbb{R}^{d_{align} \times d_k}$  为可学习的权重矩阵， $b^Q, b^K, b^V$  为相应的偏置项。本工作采用了多头注意力配置，将隐空间划分为  $h$  个独立子空间，每个子空间的维度为  $d_k = d_{align}/h$ ，这种并行注意力机制允许模型从多个表示子空间中同时捕捉流特征与包特征之间的复杂关联。

为了衡量两类特征的相关程度，通过点积运算计算  $Q$  与  $K$  在隐空间内的相似度。考虑到在高维空间下，点积结果的量级增大可能导致 Softmax 函数出现梯度消失问题。本工作采用了缩放点积处理对注意力分数进行修正，并在之后进行 Softmax 归一化，从而得到注意力权重分布  $\alpha$ ，最后利用这个权重加权求和  $V$ ，

得到融合后的特征表示，其完整的计算流如下：

$$f_{fused} = \sum \alpha_i V_i = \text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中  $QK^T$  计算了流特征与包特征在隐空间内的投影重合度。得分越高，代表包级别特征对当前业务类型的判别贡献越大。 $\text{Softmax}$  函数输出的权重分布  $\alpha$  则用于刻画包级别特征对当前特定流类型的判别贡献度。最终，通过对值向量  $V$  进行加权聚合，模型生成兼具全局规律与局部指纹的融合特征表示  $f_{fused}$ 。这种融合方式有效地缓解了由于加密导致的数据冗余问题，通过注意力权重的自动分配，模型能够忽略报头中的噪声字段，而聚焦于对分类任务真正有益的关键字段组合，并利用包级别特征的局部信息对全局的流级别特征进行细节补充，从而弥补单一模态特征在刻画复杂流量模式时表达不足的问题，提升模型的分类性能和泛化能力。

#### 4.4 深度残差分类器

经过上一节的交叉注意力模块实现跨维度的特征捕捉与融合后，系统生成的融合特征  $f_{fused}$  已具备了高度精炼的流量指纹特征。为了将这些抽象的语义特征进一步映射至最终的流量类别空间，本工作设计了一种基于深度残差网络的多层感知机架构作为主分类器。传统的线性堆叠网络在处理高维加密流量指纹时，随着网络深度的增加，容易产生的梯度消失或退化问题，为此，我们引入了跳跃连接机制，允许底层特征流通过恒等映射直接跨越多个非线性层进行传递，使得网络只需学习输入与输出之间的残差映射，而非完整的映射函数，确保了梯度流在深层网络中的高效反向传播。分类器的具体结构如图 4.4 所示。

| 阶段       | 对应代码组件                            | 功能描述                                                                                 |
|----------|-----------------------------------|--------------------------------------------------------------------------------------|
| 输入层      | <code>nn.Linear(256, 1024)</code> | 将注意力融合后的 256 维向量映射到 1024 维的高维空间，扩充特征表达。                                              |
| 残差块 (x5) | <code>ResidualBlock</code>        | <b>核心主干。</b> 每个块内包含： <code>Linear -&gt; BN -&gt; ReLU -&gt; Linear -&gt; BN</code> 。 |
| 跳跃连接     | <code>out += residual</code>      | <b>关键机制。</b> 在每个残差块内部，将输入直接加到输出上，防止梯度消失。                                             |
| 输出层      | <code>nn.Linear(1024, 6)</code>   | 将高维特征压缩至 6 个类别得分 (Logits)，对应不同的业务类型。                                                 |

图 4.4 分类器结构

#### 4.4.1 输入层

分类器的输入层用于接收来自交叉注意力机制生成的融合特征向量  $f_{fused}$ 。在进入残差主干网络之前，首先通过一个全连接层将维度从融合空间维度映射至更高维的隐藏空间，接着经过一个批量归一化层和非线性激活函数，将初步融合的特征进行非线性扩充，从而构建出表达能力更强的稀疏表征。

$$h_0 = \text{ReLU}(\text{BN}(W_{init}f_{fused} + b_{init}))$$

#### 4.4.2 深层残差主干网络

为了从融合特征中提取具有强判别力的分类特征，本工作构建了一个由多个残差块纵向堆叠而成的深层 MLP 架构作为分类网络的主干。该部分作为整个分类系统的决策核心，通过复杂的非线性演化，将融合特征映射至高维线性可分的业务空间。每个残差块采用双层线性映射结构，块内包含两次全连接映射。第一层为初步交互层，负责捕捉输入特征分量间的初步非线性交互，并将融合特征映射至更高阶的表示空间。第二层为特征精炼层，对上述高阶表示进行二次提纯，滤除噪声冗余，保留最具类别区分度的关键特征。这种双层结构模拟了小型的编码解码过程，增强了模型对复杂加密协议特征的非线性拟合精度。

为了确保深层模型在训练过程中的数值稳定性与泛化性能，在每层全连接计算之后，均嵌入了批量归一化层以稳定训练过程中的特征分布，加速模型的收敛速度。接着通过 ReLU 激活函数引入非线性变换，确保特征框架具备足够的非线性复杂度，提升模型的表达能力。此外，每个残差块内部的两个全连接层之间设置 Dropout 层，通过训练阶段随机使部分神经元失活，强迫模型学习更加独立的特征表示，从而防止出现过拟合现象，以增强模型的泛化能力。

此外，本工作构建的残差网络的核心在于引入了跳跃连接机制，在每个残差块的末端输出前，建立一条跨越两层全连接变化的恒等映射通道。使得每个残差块中第一个全连接层的输入能够直接与第二个全连接层的输出相加，从而形成跳跃连接。在反向传播过程中，由于存在跳跃连接路径，梯度可以直接通过加法算子回传至前层，确保了深层参数能够得到有效更新。这种设计能够缓解深层网络中的梯度消失问题，使得模型能够在增加网络深度的同时，保持高效的梯度流动和稳定的训练过程。设第  $i$  个残差块的输入向量为  $\mathbf{x}_i$ ，该网络的具体处理流程可描述如下：

$$\mathbf{y}_i = \sigma(\text{BN}(W_{i,1}\mathbf{x}_i + \mathbf{b}_{i,1}))$$

$$\mathcal{F}(\mathbf{x}_i) = \text{BN}(W_{i,2}\mathbf{y}_i + \mathbf{b}_{i,2})$$

$$\mathbf{x}_{i+1} = \sigma(\mathcal{F}(\mathbf{x}_i) + \mathbf{x}_i)$$

其中,  $W$  和  $b$  分别表示两个全连接层的权重和偏置, BN 表示批归一化,  $\sigma$  表示 ReLU 激活函数。  $\mathcal{F}(\mathbf{x}_i)$  代表该残差块拟合的特征映射。

#### 4.4.3 输出层逻辑

分类器的最后一层为全连接层, 其核心职能是将高度非线性的隐空间表征压缩并映射至预定义的  $n$  维业务类别空间, 每个维度分别对应一类特定的加密网络业务。输出层产生的原始逻辑得分通过 Softmax 函数执行概率标准化映射

$$P(y = k|\mathbf{x}) = \exp(z_k) / \sum_j \exp(z_j)$$

其中  $z_k$  代表第  $k$  个类别的预测得分。通过这一变换, 模型将抽象的特征分布转化为具有直观物理意义的类别概率分布, 最终选取概率峰值对应的索引作为流量分类的最终决策结果。

### 4.5 训练优化

#### 4.5.1 学习率调度

本模型的训练过程采用了基于验证集表现驱动的动态优化策略, 旨在平衡收敛速度与全局搜索能力。核心优化器选用 AdamW 算法, 利用其解耦权重衰减的特性来增强正则化效果, 并配合 OneCycleLR 学习率调度策略, 在训练初期快速爬升以越过局部最优陷阱, 并在后期执行余弦退火降温以实现参数的精确收敛。这种策略有助于模型跳出局部最优解, 在训练中寻找全局最优点。为进一步防止过拟合并提升自动化训练水平, 系统采用与上一章类似的回溯降温与早停机制: 当验证集准确率在预设的多个耐心周期内未见提升时, 训练程序将自动回溯至历史最优权重状态, 并按固定比例下调学习率。损失函数方面则统一采用交叉熵损失, 通过最小化预测概率分布与真实标签之间的 Kullback-Leibler 散度, 引导模型在迭代中不断精炼决策边界。

### 4.6 实验设计与结果分析

本文将对基于融合特征的分类模型进行全面的实验评估。在对比试验阶段, 所使用的实验环境, 数据集以及针对实验结果的评估标准与上一章一致, 在此不做过多赘述。此外, 为了验证融合特征方法相比于仅依赖流级别或包级别特征的单模态特征方法在分类性能上的优越性, 以及验证融合特征所拥有的两类特征互补优势, 我们还将通过混淆矩阵来揭示模型在各个类别上的误分类情况。

#### 4.6.1 参数设置

在网络结构层面，为了平衡特征表达能力与计算开销，特征融合空间的对齐维度  $d_{align}$  设置为 256，多头注意力机制的头数  $h$  定为 8。分类器内部通过 5 层残差块进行深层建模，每层隐藏单元的宽度保持在 1024 维，以充分容纳高维流形特征。在优化算法方面，采用 AdamW 优化器，并配置  $10^{-3}$  的初始学习率，利用 OneCycleLR 策略在 100 个 Epoch 的训练周期内执行动态调整。为抑制过拟合，模型引入了 0.2 的 Dropout 概率以及  $10^{-4}$  的权重衰减因子。此外，训练过程通过验证集性能进行实时驱动，设置早停阈值为 10，并配合学习率回溯降温机制，以确保模型在最优参数空间内平稳收敛。具体的参数设置如表 4.2 所示。

| 类别   | 参数名称                    | 配置数值               |
|------|-------------------------|--------------------|
| 架构参数 | 隐空间对齐维度 ( $d_{align}$ ) | 256                |
|      | 注意力头数 ( $h$ )           | 8                  |
|      | 残差块数量 (Blocks)          | 5                  |
|      | 隐藏层神经元数                 | 1024               |
| 优化策略 | 初始学习率 (LR)              | $1 \times 10^{-3}$ |
|      | 训练周期 (Epochs)           | 100                |
|      | 批大小 (Batch Size)        | 128                |
|      | 学习率调度器                  | OneCycleLR         |
| 正则化  | 丢弃率 (Dropout)           | 0.2                |
|      | 权重衰减 (Weight Decay)     | $1 \times 10^{-4}$ |
|      | 早停阈值 (Patience)         | 10                 |

表 4.2 模型及训练参数设置

#### 4.6.2 对比算法

为了充分验证本章所提出的基于融合特征流量分类模型在加密流量识别任务中的有效性以及相比于单模态模型的优越性，我们选择了当前领域内具有代表性的包级别特征分类算法和融合特征分类算法作为对比对象。此外我们对本章提出的模型进行了消融处理，仅使用包级别的特征作为分类特征输入到分类器中作为对比消融实验进行参考。以上的对比实验能够全面展示本模型在不同类型算法中的性能优势。具体的对比算法包括：

(1) 1D-CNN: Wang 等人提出的 1D-CNN 流量分类模型是加密流量分类领域端到端学习路径的最常用的基准模型之一，其核心逻辑在于消除对人工特征工程的依赖。该方法将原始网络流或会话的前 784 个字节视作一维序列输入。截取一个流的前 784 个字节的有效载荷，并将其映射至灰度值范围，构造成灰度图

像形式，接着利用卷积层和池化层自动、逐层地提取流量中的非线性空间特征，并最终在深度学习框架下实现从原始报文到业务类型的端到端映射与分类。

(2) **DM-HNN**: DM-HNN 模型是当前加密流量分类中特征融合路径的典型代表。其核心设计逻辑在于利用双模态特征克服单一特征源的信息缺失问题：该模型通过对门控循环单元（GRU）进行改进，在 GRU 层之间引入信息传输通道从而解决梯度回流受阻问题，并使用改进的 GRU 路径对流级别特征进行处理。接着利用堆叠自编码器（SAE）对数据包有效载荷字节进行空间建模，使用三角函数对字节的位置信息进行编码，融合将编码后的向量输入多层 SAE 中提取包级别的流量特征。最后将两种流量进行拼接，实现融合特征的提取。我们将遵循原作者的架构设置，将两路特征进行分别提取拼接后输入分类器完成分类任务。

(3) **Ours-onlyp**: 为了验证本章提出的融合机制有效性以及两种模态特征存在的互补优势。我们设计了一个仅使用本章开头提取的包级别特征作为最终分类特征的消融版本。该版本将不涉及流级别特征以及注意力融合机制，仅使用 n-gram 嵌入处理后的包级别特征作为分类器输入，同时保持分类器的各项设置不变。通过与该版本的对比，我们能够进一步量化融合特征机制在提升模型性能方面的作用。

#### 4.6.3 实验结果分析

在本节中，我们将对所提出的基于融合特征的流量分类模型与上述几种对比算法在多个数据集上的实验结果进行详细分析。具体的各项实验结果数据如表 4.3 所示，其中 Precision, Recall, F1-Score 的值为每一个类别对应值的平均数。

从整体实验数据来看，本章提出的基于融合特征的分类方法在四个公开数据集中表现出了显著的优越性。在绝大多数分类指标上均取得了最优结果。特别是在识别难度较高的 ISCX-VPNnonVPN 数据集上，本文方法的准确率达到了 97.85%，相比于基准方法 1D-CNN 提升了约 11.06%，在 F1-Score 上也保持了极高的一致性。即使在流量混淆特征极强的 VNAT 数据集中，本文方法依然能够达到 98.30% 的准确率，这充分证明了该模型在面对复杂、非对称且高度加密的网络环境时，具备极强的分类鲁棒性和泛化能力。在恶意流量数据集 USTC-TFC 上，本文方法与目前领先的双模态方法 DM-HNN 性能基本持平，准确率均突破了 99.5%，显示出该架构在处理不同性质的加密流量任务时均具有极高的适配性。

深入对比单模态与双模态特征的实验表现可以发现，多模态特征的引入是提升加密流量分类性能的关键。传统的单模态方法如 1D-CNN 仅依赖于数据包有效载荷载荷，在面对现代加密技术带来的流量随机化挑战时，往往因特征表达能力不足而陷入性能瓶颈。相比之下，DM-HNN 以及本文提出的双模态融合框

| 数据集            | 方法               | Accuracy      | Precision     | Recall        | F1-Score      |
|----------------|------------------|---------------|---------------|---------------|---------------|
| ISCX-VPNnonVPN | 1D-CNN           | 86.79%        | 86.59%        | 86.69%        | 86.60%        |
|                | DM-HNN           | 93.37%        | 93.43%        | 93.44%        | 93.31%        |
|                | Ours-onlyp       | 92.28%        | 92.31%        | 92.28%        | 92.23%        |
|                | <b>Ours</b>      | <b>97.85%</b> | <b>97.85%</b> | <b>97.86%</b> | <b>97.84%</b> |
| ISCX-TornonTor | 1D-CNN           | 88.04%        | 88.36%        | 88.05%        | 88.16%        |
|                | DM-HNN           | 91.90%        | 91.86%        | 91.95%        | 91.89%        |
|                | Ours-onlyp       | 90.81%        | 90.78%        | 90.80%        | 90.78%        |
|                | <b>BYOL-Ours</b> | <b>94.13%</b> | <b>94.14%</b> | <b>94.13%</b> | <b>94.12%</b> |
| USTC-TFC       | 1D-CNN           | 97.50%        | 97.51%        | 97.54%        | 97.50%        |
|                | DM-HNN           | <b>99.62%</b> | <b>99.62%</b> | <b>99.63%</b> | <b>99.62%</b> |
|                | Ours-onlyp       | 98.12%        | 98.18%        | 98.11%        | 98.12%        |
|                | <b>BYOL-Ours</b> | 99.51%        | 99.51%        | 99.51%        | 99.51%        |
| VNAT           | 1D-CNN           | 93.20%        | 92.80%        | 92.79%        | 92.62%        |
|                | DM-HNN           | 95.01%        | 93.95%        | 95.72%        | 94.29%        |
|                | Ours-onlyp       | 94.90%        | 95.03%        | 94.87%        | 94.80%        |
|                | <b>BYOL-Ours</b> | <b>98.30%</b> | <b>98.39%</b> | <b>98.26%</b> | <b>98.31%</b> |

表 4.3 不同数据集上的对比实验结果

架通过同时捕获时序流向的流级别特征与报文内容的包级别特征，有效地实现了信息的维度互补。实验数据清晰地表明，在所有数据集中，双模态方法的性能指标均大幅度领先于单模态方法，这印证了多模态特征挖掘能够通过捕捉更深层次的数据流行为模式，从而显著降低分类器的误报率并提升对复杂加密隧道流量的识别精度。

此外，本文设计的特征融合策略与 BYOL 自监督学习框架进一步增强了特征的判别性。通过对比同为双模态架构的 DM-HNN 可以观察到，本文方法在 VNAT 和 ISCX-VPN 等数据集上展现出了更优的特征表现，这得益于模型内部能够更深入地建模不同模态间的非线性关联。同时，消融实验结果进一步揭示了多尺度表征的重要性。实验显示，仅使用包级别特征的 Ours-onlyp 版本在各项指标上均明显低于完整版模型，例如在 ISCX-VPNnonVPN 中，完整版比消融版准确率高出 5.57%。这有力地证明了流级别与包级别特征之间存在显著的协同效应，只有通过深层的特征融合，才能在保持局部细节感知的同步实现对全局长时序行为的精准刻画，从而达成最优的分类效果。

此外，针对本章开头提到的类别混淆问题，我们还对各个类别的分类结果提取了混淆矩阵。以类别混淆问题最为严重的 VNAT 数据集为例，图 4.5 中分别

| Normalized Confusion Matrix (row-wise): |        |       |           |        |       | Normalized Confusion Matrix (row-wise): |       |       |           |        |       | Normalized Confusion Matrix (row-wise): |        |       |           |       |       |
|-----------------------------------------|--------|-------|-----------|--------|-------|-----------------------------------------|-------|-------|-----------|--------|-------|-----------------------------------------|--------|-------|-----------|-------|-------|
|                                         | chat   | file  | streaming | VoIP   | C2    |                                         | chat  | file  | streaming | VoIP   | C2    |                                         | chat   | file  | streaming | VoIP  | C2    |
| chat                                    | 100.0% | 0.0%  | 0.0%      | 0.0%   | 0.0%  | chat                                    | 86.6% | 0.0%  | 0.0%      | 4.1%   | 9.3%  | chat                                    | 100.0% | 0.0%  | 0.0%      | 0.0%  | 0.0%  |
| file                                    | 0.0%   | 99.5% | 0.5%      | 0.0%   | 0.0%  | file                                    | 0.0%  | 95.4% | 0.5%      | 1.0%   | 3.1%  | file                                    | 0.0%   | 96.4% | 3.6%      | 0.0%  | 0.0%  |
| streaming                               | 0.0%   | 0.5%  | 98.2%     | 0.9%   | 0.5%  | streaming                               | 0.9%  | 3.2%  | 95.0%     | 0.0%   | 0.9%  | streaming                               | 0.0%   | 0.0%  | 100.0%    | 0.0%  | 0.0%  |
| VoIP                                    | 0.0%   | 0.0%  | 0.0%      | 100.0% | 0.0%  | VoIP                                    | 0.0%  | 0.0%  | 0.0%      | 100.0% | 0.0%  | VoIP                                    | 0.0%   | 2.5%  | 0.0%      | 97.5% | 0.0%  |
| C2                                      | 0.0%   | 48.4% | 1.1%      | 0.0%   | 50.5% | C2                                      | 1.1%  | 0.5%  | 0.0%      | 1.1%   | 97.4% | C2                                      | 0.5%   | 0.5%  | 1.1%      | 0.5%  | 97.4% |

图 4.5 三种模型的混淆矩阵

为：（1）仅使用前 16 个数据包长度序列作为原始特征输入 BYOL 模型的流级别特征分类结果。（2）仅使用包级别特征作为分类特征的分类结果。（3）使用融合特征作为最终分类特征的分类结果。

仅使用流级别特征的方法在对常规业务如 chat 和 VoIP 具有极高的识别率，但在处理 C2 流量时存在严重的误判现象约有 48.4% 的 C2 流量被错误地识别为 file 传输。这表明单一的流级别长度序列在面对具有相似突发性行为的加密流量时，容易产生特征重叠，无法有效提取区分命令控制与文件传输的微观指纹。而仅使用包级别特征的分类结果，在 C2 流量上的识别精度有了显著提升，但是在 chat 类别上却表现出性能退化，有约 9.3% 的 chat 流量被误分类为 C2。这说明虽然包级别的深层特征对于捕捉特定的应用层行为非常有效，但在缺乏宏观流行行为的约束下，容易将一些短小频繁的即时通讯包误判为控制指令。

而使用融合特征的分类模型有效地结合了两者的优势，补齐了单一模态的性能短板。在融合特征下，chat 和 streaming 的识别率均达到了 100%，同时 C2 流量的识别率维持在 97.4% 的高水准，且原本在包级别特征中出现的 chat 误判为 C2 的现象得到了根本性抑制。这组对比实验有力地证明了：流级别特征提供了宏观的业务框架约束，而包级别特征提供了微观的业务指纹校验，两者的深度融合能够显著消除特定类别间的混淆问题，使模型在多业务共存的复杂网络流量中展现出更精准的分类性能。

## 4.7 本章小结

## 第 5 章 总结与展望

### 5.1 本文工作总结

### 5.2 未来展望

## 第 6 章 引用文献的标注

模板使用 **natbib** 宏包来设置参考文献引用的格式，更多引用方法可以参考该宏包的使用说明。

### 6.1 顺序编码制

#### 6.1.1 角标数字标注法

|                                       |               |                    |
|---------------------------------------|---------------|--------------------|
| <code>\cite{knuth86a}</code>          | $\Rightarrow$ | [? ]               |
| <code>\citet{knuth86a}</code>         | $\Rightarrow$ | ?]                 |
| <code>\cite[42]{knuth86a}</code>      | $\Rightarrow$ | [? ] <sup>42</sup> |
| <code>\cite{knuth86a,tlc2}</code>     | $\Rightarrow$ | [? ? ]             |
| <code>\cite{knuth86a, knuth84}</code> | $\Rightarrow$ | [? ? ]             |

#### 6.1.2 数字标注法

|                                       |               |                    |
|---------------------------------------|---------------|--------------------|
| <code>\cite{knuth86a}</code>          | $\Rightarrow$ | [? ]               |
| <code>\citet{knuth86a}</code>         | $\Rightarrow$ | ? ]                |
| <code>\cite[42]{knuth86a}</code>      | $\Rightarrow$ | [? ] <sup>42</sup> |
| <code>\cite{knuth86a,tlc2}</code>     | $\Rightarrow$ | [? ? ]             |
| <code>\cite{knuth86a, knuth84}</code> | $\Rightarrow$ | [? ? ]             |

### 6.2 著者-出版年制标注法

|                                       |               |                   |
|---------------------------------------|---------------|-------------------|
| <code>\cite{knuth86a}</code>          | $\Rightarrow$ | ?                 |
| <code>\citep{knuth86a}</code>         | $\Rightarrow$ | (?)               |
| <code>\citet[42]{knuth86a}</code>     | $\Rightarrow$ | ? <sup>42</sup>   |
| <code>\citep[42]{knuth86a}</code>     | $\Rightarrow$ | (?) <sup>42</sup> |
| <code>\cite{knuth86a,tlc2}</code>     | $\Rightarrow$ | ??                |
| <code>\cite{knuth86a, knuth84}</code> | $\Rightarrow$ | ??                |

## 致谢

在研究学习期间，我有幸得到了三位老师的教导，他们是：我的导师，中国科大 XXX 研究员，中科院 X 昆明动物所马老师以及美国犹他大学的 XXX 老师。三位深厚的学术功底，严谨的工作态度和敏锐的科学洞察力使我受益良多。衷心感谢他们多年来给予我的悉心教导和热情帮助。

感谢 XXX 老师在实验方面的指导以及教授的帮助。科大的 XXX 同学和 XXX 同学参与了部分试验工作，在此深表谢意。

## 在读期间取得的科研成果

### 已发表论文：

- [1] xx. 中国科学技术大学研究生学位论文撰写规范. 中国科学技术大学学报, 2024,1.
- [2] .....

### 发明专利：

- [1] xx. 一种温热外敷药制备方案. 授权号：CN123456789B, 2023-08-12.
- [2] .....

### 会议论文：

- [1] xx. 关于人工智能在学位授予工作的应用, 人工智能国际研讨会, 2023,12.
- [2] .....

### 参与的科研项目：

- [1] 智能计算板卡与整机, 科技委重点项目-课题, 2018-2020.
- [2] .....